

Optimization Techniques for Multi Cooperative Systems MCTR 1021
Mechatronics Engineering
Faculty of Engineering and Materials Science
German University in Cairo

Optimising Cooperative Spacecraft Configurations for Asteroid Surface Coverage

Milestone 4: Genetic Algorithm

By

Team 21

Omar Fahmy (58-10021)
Yassin Mohamed (58-16061)
Omar Alaa (58-13517)
Mohamed Rashad (58-7580)
Alaa Abdelnaser (58-22615)
Malak Hisham (58-0665)

Course Team

Dr. Eng. Omar M. Shehata
Eng. Mohamed A. Ibrahim
Eng. Abdelrahman Yasser
Eng. Rodaina Raed
Eng. Ahmed Mostafa

19 October 2026

This is to certify that:

(i) the report comprises only our original work toward the course project,

(ii) due acknowledgment has been made in the text to all other material used

Team 21
19 October 2026

Acknowledgments

We would like to thank the MCTR 1021 course team for their guidance throughout this project.

Abstract

This report presents the Genetic Algorithm (GA) milestone of our project on cooperative spacecraft fleets that photograph the surface of an irregular asteroid, and compares it with the Simulated Annealing (SA) of Milestone 3. The optimiser chooses how many spacecraft to send (one on/off switch per slot), the circular orbit of each and the mission duration; the fitness rewards the coverage of the sunlit surface and penalises fleet cost, energy and mission time, under orbit-radius, separation and battery constraints. Every candidate is scored by a true-scale mission simulation with Kepler orbits, a spinning non-convex shape model, ray-cast occlusion, sunlight, a camera resolution limit and batteries. The GA is written from scratch on the same Algorithm $\rightarrow$ Feasibility $\rightarrow$ Engine pipeline. Its chromosome is the whole mission, one five-gene slot per spacecraft. Problem-specific operators align the slots of two parents before crossover (a fleet can be stored in $N_{\max}!$ orders), swap whole spacecraft or blend matched orbits with BLX-α along the shorter arc for the angles, start from a population stratified over the fleet size, and reuse SA's move as mutation. A parameter study of 23 settings confirmed every default (population 40 or 60, 5 % elites, crossover fraction 0.8, binary tournament, blend probability 0.3) and showed that both crossover and mutation are needed and that midpoint averaging is the worst choice ($p = 0.004$). The GA finds the known optima of benchmark functions in every run and, under the assumptions of the literature, images 97.7 % of Eros (100 % reported there with a simpler visibility model). On nine case studies (26 or 51 genes, 2000 evaluations, five seeds) SA is slightly better (mean fitness 0.2663 against 0.2734, significant in 2 cases), while the GA is on a par in 7 cases, varies less between runs and has the lower mean fitness on the two-hour Eros mission; with 5000 evaluations it becomes significantly better there ($p = 0.048$). On a matched 51-gene Eros case (1200 evaluations, three seeds) SA reaches $J = 0.1089$ against $J = 0.1237$ for the GA, with mean coverage of 94.44 % against 93.53 %. All final designs are feasible; hybrid local refinement is the clearest route to improving the GA.

Contents

Acknowledgments **iii**

List of Abbreviations **viii**

Nomenclature **ix**

List of Figures **xi**

List of Tables **xiii**

1 Introduction **1**
1.1 Objectives 1
1.2 What changed since Milestone 2 2
1.3 Report outline 2

2 Literature Review **3**
2.1 Swarm co-orbits for planetary moon mapping 3
2.2 Swarm flyby mapping of asteroid Eros 3
2.3 Multi-mission CubeSat constellation design 3
2.4 Walker constellation design for LEO navigation 4
2.5 Swarm orbital formation around asteroids 4

3 Problem Formulation **6**
3.1 Problem statement 6
3.2 Type of problem 7
3.3 Decision variables 8
3.4 Objectives 8
3.5 Fitness function 9
3.6 Constraints 10
3.7 Parameters 10
3.8 Assumptions 11

4 Simulation Model **13**
4.1 Asteroid mesh . . . 14
4.2 Stage 1: orbit propagation . . . 14
4.3 Stage 2: rotation into the asteroid frame . . . 15
4.4 Stage 3: which facets are photographed . . . 15
4.5 Stage 4: ΔV, power and battery . . . 15
4.6 Making the simulation fast enough . . . 16

5 Simulated Annealing Implementation **17**
5.1 Code structure: Algorithm $\rightarrow$ Feasibility $\rightarrow$ Engine . . . 17
5.2 The algorithm . . . 18
5.3 Problem-specific considerations . . . 18
5.4 Algorithm parameters . . . 21
5.5 Visualisation of the search and of the solutions . . . 21

6 Case Studies **24**

7 Validation **26**
7.1 V1: the simulator obeys physics . . . 26
7.2 V2: SA finds known optima . . . 27
7.3 V3: is there anything to optimise? . . . 27
7.4 V4: SA against random search . . . 28
7.5 V5: comparison with the literature . . . 30
7.6 V6: is the chosen fleet size the best? . . . 31
7.7 V7: finer asteroid model . . . 32
7.8 Summary of the validation . . . 32

8 Results and Discussion **33**
8.1 Fleets designed for every case study . . . 33
8.2 The best designs at work . . . 34
8.3 What the optimiser learned . . . 36
8.4 Algorithm performance . . . 37
8.5 Effect of the algorithm parameters . . . 38
8.6 Recommendations . . . 39

9 Genetic Algorithm Implementation **41**
9.1 Chromosome representation . . . 41
9.2 Population size . . . 42
9.3 Initial population . . . 45
9.4 Selection . . . 46
9.5 Population split: elites, crossover and mutation . . . 46

9.6 Crossover . . . 49
9.6.1 Step 1: slot alignment . . . 49
9.6.2 Step 2: swap or blend each aligned pair . . . 50
9.6.3 Choice of the blend probability . . . 50
9.7 Mutation . . . 52
9.8 Constraints, duplicates and termination . . . 53
9.9 One algorithm for the fleet and for benchmark functions . . . 53
9.10 The complete algorithm . . . 54

10 Genetic Algorithm: Validation, Results and Comparison with SA **57**
10.1 Case studies and input parameters . . . 57
10.2 Validation . . . 58
10.3 Parameter study . . . 59
10.4 Results of the case studies . . . 61
10.5 Harder tests: large body, short missions, larger budget . . . 62
10.6 Comparison of SA and GA on the case studies . . . 64
10.7 Matched comparison on a larger Eros case . . . 66
10.7.1 Evidence across the complete comparison set . . . 67
10.7.2 Why mutation and crossover are different . . . 67
10.8 Recommendations . . . 69

11 Conclusion **70**

12 Future Work **72**

Appendix **74**

A Running the Code **75**

B Decision Vector Layout **77**

References **78**

List of Abbreviations

GA	Genetic Algorithm
NSGA	Non-dominated Sorting Genetic Algorithm
PSO	Particle Swarm Optimisation
SA	Simulated Annealing

Nomenclature

Symbol	Meaning	Unit
Decision variables		
$\mathbf{x}$	decision vector, $\dim \mathbf{x} = 1 + 5N_{\max}$	–
T	mission duration	d
a_i	orbit radius of slot i (circular orbit)	km
ι_i	inclination of orbit i	deg
Ω_i	longitude of the ascending node of orbit i	deg
$M_{0,i}$	mean anomaly (start position) of spacecraft i at $t = 0$	deg
s_i	on/off switch of slot i	–
N, $N_{\max}$	number of flying spacecraft, number of slots	–
Objectives, fitness and constraints		
C	coverage: imaged share of the sunlit area	–
S_a, A^*	area imaged within T, area sunlit at some time within $T_{\max}$	km^2
Z	fleet cost	–
E, ΔV_i, $E_{\mathrm{elec},i}$	energy term, velocity change and electrical energy of spacecraft i	–, m/s, J
J	fitness (minimised)	–
w_C, w_Z, w_E, w_T	weights of coverage, cost, energy and time	–
$g_1 \dots g_4$	normalised constraints (radius, radius, separation, battery)	–
J_0, ρ	base cost and penalty coefficient of infeasible designs	–
$T_{\min}$, $T_{\max}$	shortest and longest allowed mission	d
$r_{\min}$, $r_{\max}$	smallest and largest orbit radius	km
κ	safety factor, $r_{\min} = \kappa R_{\max}$	–
d_{sep}	minimum distance between spacecraft	km
q_i, $q_{\min}$	battery state of charge of spacecraft i, its lower limit	–
n_{ref}	reference fleet size for normalisation (10)	–
Model		
μ	gravitational parameter of the asteroid	km^3/s^2
P_{rot}, ω_{ast}	rotation period and spin rate of the asteroid	h, rad/s
$R_{\max}$	distance of the farthest surface point from the centre	km
f, A_f, $\mathbf{c}_f$, $\mathbf{n}_f$	facet index, area, centre and outward normal	–, km^2, km, –
t_k, Δt, N_t	time step k, step length, number of steps	s, min, –
n_i, u_i	mean motion and argument of latitude of spacecraft i	rad/s, rad
$\mathbf{r}_i^I$, $\mathbf{r}_i^B$	position of spacecraft i in the inertial and body frame	km
ψ, p	camera field of view and pixels across	deg, –
ℓ, $\ell_{\max}$	range facet–spacecraft and usable camera range	km

Symbol	Meaning	Unit
GSD	ground sample distance (size of one pixel on the ground)	m
$\eta_{\max}$	largest emission angle	deg
V_{ifk}, u_f	facet f imaged by spacecraft i at step k; imaged by any spacecraft	–
$n_{\mathrm{img},i}$	number of frames taken by spacecraft i	–
r_0, v_0	radius and speed of the arrival orbit	km, km/s
Simulated Annealing		
Θ, Θ_0, $\Theta_{\min}$	temperature, start and final temperature	–
α	geometric cooling factor	–
L	moves per temperature (Markov chain length)	–
p_0	initial acceptance probability of a typical worse move	–
σ, σ_0	move size and initial move size (fraction of a variable's range)	–
B, n_{init}	evaluation budget, random designs of the warm start	–

List of Figures

3.1 The two test bodies drawn at the same scale: 433 Eros (Gaskell shape model, 33 km long) and the dog-bone shaped 216 Kleopatra (radar shape model, 219 km long). 7

4.1 The simulator at true scale: the best design of case A2 (four spacecraft, Eros) after 4.4 h, with the orbits and the 6° camera cones; green facets are already photographed, dark blue facets are never sunlit during the mission, the yellow line points to the Sun. 13
4.2 The asteroid as a triangle mesh: every facet is one small flat piece of the surface. 14

5.1 The optimisation pipeline. The algorithm proposes a decision vector; the feasibility layer rejects (with a penalty) or passes it; the engine simulates the mission and returns the fitness, which is fed back to the algorithm. 17
5.2 Flowchart of the Simulated Annealing implementation. 20
5.3 The viewer while Simulated Annealing (SA) runs: 3D scene with the current fleet (left), objectives, constraints and optimiser settings (right), live charts of coverage, fitness, designs tried and temperature (bottom). 23

7.1 V3: coverage reached by random designs after ten days on Eros. 28
7.2 V4: best fitness of SA and of random search with the same budget (bars: means, dots: single runs; lower is better). 29
7.3 V6: best fitness when the fleet size is forced to N (line) and the free-N SA runs (stars, at the N they chose). 31

8.1 Eros: fleet size, mission duration and coverage chosen by SA against the longest allowed mission $T_{\max}$ (each marker is one run). 34
8.2 Case A1 (Eros, $T_{\max} = 10\,\mathrm{d}$): the best design at four moments of its mission. . 35
8.3 Case A5 (Eros, $T_{\max} = 6\,\mathrm{h}$): the best design at four moments of its mission. . . 35
8.4 Case B1 (Kleopatra, $T_{\max} = 3\,\mathrm{d}$): the best design at four moments of its mission. 35
8.5 Orbits of every spacecraft in the best feasible SA designs: radius in units of the farthest surface point (dotted line: $r_{\min} = 1.5R_{\max}$) and inclination. 36

8.6 Convergence: best fitness against evaluations, mean ± one standard deviation over 5 runs, for SA and random search in every case of A and B. 37
8.7 Case E: final best fitness for each parameter value (bars: means, dots: runs; blue: the default). 38
8.8 Case E: mean convergence for each parameter value. 39

9.1 Chromosome representation of the Genetic Algorithm (GA). (a) A gene is one number, a slot is the five genes of one spacecraft, and the chromosome is the whole decision vector (Eq. 3.1): the mission duration followed by $N_{\max}$ slots. In this example ($N_{\max} = 5$) slots 1, 2 and 4 fly ($N = 3$); the orbit genes of slots 3 and 5 are dormant but still inherited. (b) The population holds P such chromosomes, each a complete mission with its own fleet size. 43
9.2 Assembly of a new generation ($P = 40$, defaults $\varepsilon = 5\,\%$, $\chi = 0.8$). The two best designs survive unchanged; every other place is filled by a new child, bred from parents chosen by binary tournament among *all* P individuals. 47
9.3 Slot crossover of two 3-spacecraft parents ($N_{\max} = 5$; the duration T is left out for clarity). Parent B stores its orbits in a different slot order; without alignment, slot 1 of A (50 km, 30°) would be paired with slot 1 of B (41 km, 140°), an unrelated orbit. After alignment each column holds two similar orbits. Here the first pair is blended (the nodes 350° and 10° give 0° along the shorter arc) and the others are swapped. Blue: from A; orange: from B; mixed: blended; grey: switched off. 52
9.4 Flowchart of the Genetic Algorithm. Every evaluation passes through the same Feasibility → Engine pipeline as SA. 55

10.1 Parameter study of the GA (case F). Dots: best design of each of the five runs; bar: mean. Dashed line: the default settings. 60
10.2 Best fitness of every run (dots) and mean (bar) for SA, GA and random search, budget 2000 evaluations, five seeds per algorithm and case. 62
10.3 Convergence: median over the seeds of the best fitness found so far, with the interquartile band. SA's curve starts after its warm start (the first 40 evaluations are not recorded). 63
10.4 The GA run with the median result on Eros 10 d, generation by generation: fitness, diversity of the active genes, fleet sizes in the population; right: the operator that produced each new best design, over all 45 case-study runs. 63
10.5 Left: hard cases H1–H3, best fitness of every run and mean. Right: the same seeds with a budget of 2000 (open markers) and 5000 evaluations (filled); the arrow joins the means. 65

List of Tables

2.1 Summary of the reviewed formulations 4

3.1 Classification of the optimisation problem 7
3.2 Decision variables. Every orbit is circular ($e = 0$), so the argument of periapsis has no meaning; the radius a_i is the orbit's semi-major axis. 8
3.3 Constraints. Index i runs over the flying spacecraft ($s_i = 1$), $t_k = k\,\Delta t$ over the time steps of the mission. 10
3.4 Mission and problem parameters 11

5.1 Main code modules (package `spacecov`) 18
5.2 Problem-specific ingredients of the SA implementation 22
5.3 Default SA parameters 22

6.1 Case studies and their input parameters (budget 2000 evaluations per run) . . . 25
6.2 Size of the case studies and the cost of one SA run (2000 evaluations, one CPU core; nine single-threaded runs in parallel on one machine) 25

7.1 Validation plan 26
7.2 V1: physics checks (selection) 27
7.3 V2: SA on test functions with known minimum 0 (10 runs of 10 000 evaluations) 27
7.4 V3: coverage of random designs flown for ten days around Eros 28
7.5 V4: SA (bold) against random search with the same budget; means over 5 runs each 29
7.6 V5: comparison with the closest prior work on Eros 30
7.7 V6: forced fleet size against the fleet size SA chooses itself 31

8.1 Fleets designed by SA in every case study (mean over runs; N is listed per run) 33
8.2 Case E: effect of the SA parameters (Eros, $T_{\max} = 2\,\mathrm{d}$, $N_{\max} = 5$; one parameter changed at a time from the defaults) 38

9.1 Population size and number of generations for the budget $B = 2000$ used by SA, with the default elite share $\varepsilon = 5\,\%$ (Section 9.5). 43

9.2 Expected number of individuals per fleet size N in a uniformly random initial population, $P \cdot \Pr(N)$ with $N \sim \mathrm{Binomial}(N_{\max}, \frac{1}{2})$ and an all-off draw counted as $N = 1$. 44
9.3 Number of individuals per fleet size N in the first generation (uniform: expected value). 45
9.4 Group sizes of Eq. (9.3) for the default $\varepsilon = 5\,\%$, $\chi = 0.8$, and the number of generations G (Eq. 9.2) for $B = 2000$. 47
9.5 Expected number of times an individual of rank r is chosen as a parent in one generation, $(P - n_e)\Pr(r)$ with Eq. (9.4), for $P = 40$, $n_e = 2$. 48
9.6 Values of the elite share and the crossover fraction tested in the parameter study (one parameter changed at a time; default in bold). 49
9.7 Effect of the blend probability p_b on a crossover child with $N = 3$ matched flying pairs. The values marked $\star$ are tested in the parameter study, together with the fixed weight $\lambda = \frac{1}{2}$ and with the alignment switched off. 51
9.8 Parameters of the GA: default values and the values compared in the parameter study (one parameter changed at a time; a value replaces the default only if it is significantly better, Section 10.3). 56

10.1 Additional case studies of this milestone (mission settings as in Table 6.1; $T_{\min} = \min(6\,\mathrm{h}, T_{\max}/4)$; five seeds per algorithm, three for case M). 58
10.2 Known-optimum test (minimum 0): best value reached, 10 seeds $\times$ 10 000 evaluations. 58
10.3 Parameter study (case F, five seeds per setting). Tracks: spacecraft actually simulated per run (computing cost independent of the machine). 59
10.4 Hard cases H1–H3 (2000 evaluations, five seeds): mean fitness, coverage and fleet size; p: one-sided Mann–Whitney test that the GA is better. 64
10.5 Mean fitness with 2000 and 5000 evaluations (same seeds); p: one-sided Mann–Whitney tests at 5000 evaluations (bold: $p < 0.05$). 64
10.6 SA against GA on the nine case studies (2000 evaluations, five seeds, means). Bold: significantly better ($p < 0.05$). Evals to target: median evaluations until the best fitness reaches the worse of the two median final values. 65
10.7 Matched comparison of GA and SA on Eros (case M, 1200 evaluations each). Lower J is better; all final designs are feasible. Runtime: wall-clock seconds of one run with nine runs in parallel on the same machine. 67
10.8 Complete set of GA–SA comparisons. 68

A.1 Commands 76

B.1 Layout of the decision vector $\mathbf{x}$ (Eq. 3.1); slot $i = 1 \ldots N_{\max}$ 77

Chapter 1

Introduction

Asteroids are leftovers from the formation of the Solar System. Before a spacecraft can land on one or take a sample, scientists need a detailed photographic map of its surface. Most small bodies are irregular, non-convex shapes whose ridges and craters hide parts of the surface from view, and mapping them takes a long time: NEAR Shoemaker, the only spacecraft that ever orbited 433 Eros, spent about a year lowering its orbit step by step to map the body with a single camera [1].

This project asks how a small fleet of *cooperating* spacecraft should be designed so that together they photograph as much of an irregular body as possible, without spending more spacecraft, energy or mission time than needed. Milestone 2 reviewed the literature and formulated the problem. Milestone 3 implemented Simulated Annealing (SA) for it, applied it to case studies of increasing size and difficulty, and validated the results. This milestone (Milestone 4) adds a Genetic Algorithm (GA) on the same pipeline, validates it in the same way and compares it with SA. The Milestone 3 chapters are kept, updated to the final code, because the GA is measured against them.

1.1 Objectives

The objectives of Milestones 3 and 4, following the project brief, are:

1. Finalise the problem formulation (problem statement, decision variables, constraints and fitness) and implement it as a modular simulation pipeline (Chapters 3 and 4).
2. Implement SA for the problem, including its problem-specific neighbourhood and repair operators, and visualise the search and the resulting solutions (Chapter 5).
3. Define case studies of varying size and complexity and their input parameters (Chapter 6).

4. Validate the algorithm on benchmark problems with known optima, against random search, against results from the literature, and with independent checks of its decisions (Chapter 7).
5. Report and discuss the results of every case study, and analyse the performance of the algorithm and the effect of its parameters (Chapter 8).
6. **Milestone 4:** implement a GA with a problem-specific chromosome, selection, crossover and mutation (Chapter 9), validate it, study its parameters, and compare it with SA on the same case studies and budgets (Chapter 10).

1.2 What changed since Milestone 2

- Only the orbital layer is optimised. Orbits are circular (eccentricity and argument of periapsis removed) and every spacecraft carries the same narrow camera (scanner type removed).
- The mission duration T is a decision variable that is minimised, instead of a fixed three days, and the number of spacecraft N is decided by the optimiser through one on/off switch per spacecraft slot.
- The model now includes sunlight with cast shadows, eclipses and a battery constraint, and the objectives are coverage, fleet cost, energy and mission time.
- The method was extended to a second, very different body, 216 Kleopatra.

1.3 Report outline

Chapter 2 summarises the literature reviewed in Milestone 2. Chapter 3 gives the final problem formulation and Chapter 4 the simulation that evaluates it. Chapter 5 explains the SA implementation, Chapter 6 the case studies, Chapter 7 the validation and Chapter 8 the results and the analysis of the algorithm. Chapter 9 explains the GA implementation and Chapter 10 its validation, parameter study and comparison with SA. Chapters 11 and 12 conclude and outline the next milestones. Appendix A explains how to run the code.

Chapter 2

Literature Review

This chapter reviews five works (2019–2023) that formulate a multi-spacecraft coverage problem and solve it with a meta-heuristic: a Genetic Algorithm (GA), Particle Swarm Optimisation (PSO), Non-dominated Sorting Genetic Algorithm (NSGA)-II, NSGA-III and Simulated Annealing (SA). Table 2.1 summarises their formulations.

2.1 Swarm co-orbits for planetary moon mapping

Nallapu and Thangavelautham [2] design swarms that map the moon Deimos. The variables mix integers (number of encounters, spacecraft per encounter) with reals (encounter angles). The objective is $\min N$ subject to coverage $\geq 90\,\%$, no collisions and a fuel budget. A GA is compared with PSO, which uses a penalty for infeasible designs. GA found 3 spacecraft for 90.5 % coverage, PSO found 4. *Strength:* a fair GA/PSO comparison. *Shortcoming:* only one objective, and hidden terrain is not checked.

2.2 Swarm flyby mapping of asteroid Eros

Nallapu and Thangavelautham [3] place N spacecraft on a ring around Eros. The variables are $N \in [1, 20]$ and each spacecraft's angle $\theta_i \in [0, 2\pi]$; the problem is $\min N$ subject to $100\,\%$ surface coverage. A mixed-integer GA found that 8 spacecraft cover all of Eros in one pass. *Shortcoming:* the spacecraft only point at the centre, lighting and blocked views are ignored.

2.3 Multi-mission CubeSat constellation design

Melaku and Kim [4] optimise a CubeSat constellation using inclination i, number of planes P and satellites per plane S. They minimise PS and the average revisit time, and maximise

coverage, with NSGA-II (population 100, 500 generations). For example, 20 satellites give 98.8 % coverage. *Strength:* a Pareto front of trade-offs. *Shortcoming:* only three variables, and the Earth is a sphere.

2.4 Walker constellation design for LEO navigation

Guan et al. [5] design navigation constellations with variables S, P, i and altitude. They minimise the worst-case positioning error (GDOP) and the number of satellites with NSGA-III, and use a GA for a single-objective case. For example, 180 satellites at 1500 km give GDOP < 3. *Strength:* a worst-case objective and parallel computing for slow evaluations. *Shortcoming:* navigation rather than imaging.

2.5 Swarm orbital formation around asteroids

Stolfi Rosso and Danoy [6] spread spacecraft evenly around an asteroid to maximise coverage while saving fuel. A GA tunes the formation parameters for 2, 3, 5 and 10 satellites, and SA tunes each spacecraft on board; SA was more accurate over 400 scenarios. *Strength:* shows SA works for this kind of problem. *Shortcoming:* even spacing is used instead of measured coverage.

Table 2.1: Summary of the reviewed formulations

Work	Decision variables	Objectives	Algorithm	Key result
[2] 2021	encounters, spacecraft per encounter, angles	min N s.t. coverage, collision, fuel	GA, PSO	3 S/C, 90.5 %
[3] 2019	N, ring angles θ_i	min N s.t. 100 % coverage	GA	8 S/C, 100 % of Eros
[4] 2023	i, P, S	min PS, min revisit, max coverage	NSGA-II	20 S/C, 98.8 %
[5] 2020	S, P, i, altitude	min worst GDOP, min SP	NSGA-III, GA	180 S/C, GDOP $<$ 3
[6] 2023	formation parameters	even spread, fuel	GA, SA	SA best in 400 cases

Metrics and gap. From these works we take coverage as the main objective, the number of spacecraft and the fuel as costs, and the best, mean and standard deviation of the fitness over repeated runs to compare algorithms. *Gap:* the reviewed works either fix the trajectories (a single flyby or a ring), treat the body as convex or fully lit, or optimise the number of spacecraft separately from their orbits. Our formulation decides the fleet size, every orbit and the

mission duration together, and measures coverage on the real non-convex shape with occlusion, sunlight, cast shadows and an image-resolution limit. The comparison with Nallapu and Thangavelautham [3] is repeated with our model in Section 7.5.

Chapter 3

Problem Formulation

Milestone 2 formulated the problem in two layers, a static layer (fixed spacecraft) and an orbital layer (spacecraft on Keplerian orbits). This milestone optimises the orbital layer only, with three changes agreed by the team: the mission duration T becomes a decision variable instead of a constant, the number of spacecraft N is decided by the optimiser, and every orbit is circular. This chapter gives the final formulation that both the SA algorithm of Chapter 5 and the GA of Chapter 9 solve. The simulation that turns a design into the quantities used here is described in Chapter 4.

3.1 Problem statement

A fleet of up to $N_{\max}$ identical spacecraft is sent to an irregular, rotating asteroid. Every spacecraft flies its own circular orbit and carries the same narrow-angle camera. The task is to decide *how many* spacecraft to send, *which orbit* each one flies and *how long* the mission lasts, so that together the spacecraft photograph as much of the sunlit surface as possible with a usable image resolution, while sending as few spacecraft as possible, using as little energy as possible and finishing as early as possible. The orbits must stay at a safe distance from the asteroid and from each other, and no battery may run down.

The problem is *cooperative*: a surface facet that one spacecraft has already photographed adds nothing to the coverage when another spacecraft photographs it again, so the value of a spacecraft depends on what the rest of the fleet sees. The method only needs a triangle mesh of the body, its gravitational parameter and its spin; it is applied to two real asteroids, 433 Eros and 216 Kleopatra (Figure 3.1).

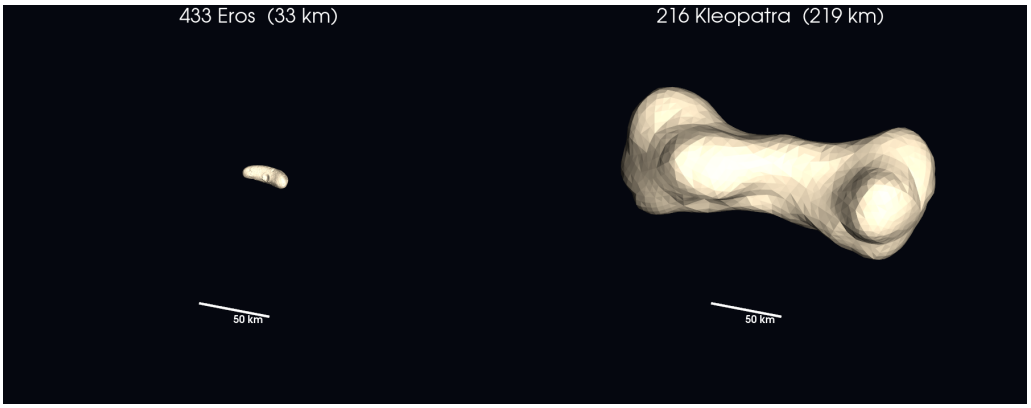


Figure 3.1: The two test bodies drawn at the same scale: 433 Eros (Gaskell shape model, 33 km long) and the dog-bone shaped 216 Kleopatra (radar shape model, 219 km long).

3.2 Type of problem

A solution is a vector of numbers, so the problem is an *arithmetic* problem (Table 3.1). It is strictly *mixed-integer*: the orbit variables and the duration are real numbers, while one on/off switch per spacecraft slot is binary. It is not a permutation problem, because exchanging two spacecraft gives the same fleet.

Table 3.1: Classification of the optimisation problem

Type	A solution is …	Our problem?
Permutation	an ordering of items	No: nothing is ordered; swapping two spacecraft gives the same fleet.
Binary	a string of 0/1 choices	Partly: the slot switches s_i are 0/1, the orbits are not.
Arithmetic	a vector of real or integer numbers	Yes: T, a_i, ι_i, Ω_i, $M_{0,i}$ are real, s_i is integer (mixed-integer).

Three properties make the problem hard for an optimiser and motivate a meta-heuristic:

- **Black box.** The fitness comes from a full mission simulation (orbits, rotation, ray casting, sunlight, batteries); there is no formula to differentiate.
- **Flat regions.** An orbit beyond the camera's usable range photographs nothing, so many poor designs share almost the same fitness.
- **Variable size.** The number of flying spacecraft is itself a decision, so part of the vector is ignored whenever a slot is switched off.

3.3 Decision variables

The decision vector has one head variable, the mission duration T, followed by $N_{\max}$ *slots* of five variables each (Table 3.2):

$$\mathbf{x} = \left[\, T,\ \mathbf{y}_1,\ \ldots,\ \mathbf{y}_{N_{\max}} \,\right], \qquad \mathbf{y}_i = \left[a_i,\ \iota_i,\ \Omega_i,\ M_{0,i},\ s_i\right] \text{ (slot } i\text{)}, \qquad \dim \mathbf{x} = 1 + 5N_{\max} \tag{3.1}$$

Slot i flies a spacecraft when its switch $s_i = 1$; the number of spacecraft is therefore

$$N = \sum_{i=1}^{N_{\max}} s_i, \qquad 1 \le N \le N_{\max}. \tag{3.2}$$

The orbit numbers of a switched-off slot are carried along unused. A fixed-length vector lets every algorithm (SA and GA now, PSO later) treat a design as one point in a box, while the optimiser still decides the fleet size. With $N_{\max} = 5$ the vector has 26 entries, with $N_{\max} = 10$ it has 51.

Table 3.2: Decision variables. Every orbit is circular ($e = 0$), so the argument of periapsis has no meaning; the radius a_i is the orbit's semi-major axis.

Symbol	Meaning	Type	Domain
T	mission duration	continuous	$[T_{\min}, T_{\max}]$ (days)
a_i	orbit radius of slot i	continuous	$[r_{\min}, r_{\max}]$ (km)
ι_i	inclination (tilt of the orbit to the asteroid equator)	continuous	$[0°, 180°]$
Ω_i	longitude of the ascending node (where the orbit crosses the equator)	continuous, periodic	$[0°, 360°)$
$M_{0,i}$	mean anomaly at $t = 0$ (where the spacecraft starts on its orbit)	continuous, periodic	$[0°, 360°)$
s_i	slot switch: 1 = the slot flies a spacecraft, 0 = unused	binary	$\{0, 1\}$

Compared with Milestone 2, the eccentricity e_i, the argument of periapsis ω_i and the scanner type t_i were removed (circular orbits, one camera for all spacecraft), and T and the switches s_i were added.

3.4 Objectives

Four quantities describe a design. Coverage and mission time lie in $[0, 1]$; fleet cost and energy use a fixed ten-spacecraft reference and may exceed one for fleets larger than that reference.

Coverage C (maximised). Let $u_f = 1$ if facet f of area A_f was photographed by at least one spacecraft within the mission, under all the image-quality rules of Section 4.4. Only facets that the Sun lights at some time can ever be photographed, so coverage is measured against the

sunlit area A^*:

$$C = \frac{S_a}{A^*}, \qquad S_a = \sum_f A_f \, u_f, \qquad A^* = \sum_{f\text{: lit at some } t \le T_{\max}} A_f. \tag{3.3}$$

Fleet cost Z (minimised). Each spacecraft costs a bus (1.0) plus its camera (0.2). The total is divided by the cost of a reference fleet of $n_{\text{ref}} = 10$ spacecraft, so

$$Z = \frac{N\,(c_{\text{bus}} + c_{\text{cam}})}{n_{\text{ref}}\,(c_{\text{bus}} + c_{\text{cam}})} = \frac{N}{10}. \tag{3.4}$$

A fixed reference (instead of $N_{\max}$) makes one spacecraft cost the same in every case study, so cases with different $N_{\max}$ are comparable and oversized fleets remain penalised.

Energy E (minimised). Half propellant, half electricity:

$$\begin{aligned} E &= \tfrac{1}{2}\,\frac{\sum_i \Delta V_i}{n_{\text{ref}}\, 3v_0} + \tfrac{1}{2}\,\frac{\sum_i E_{\text{elec},i}}{n_{\text{ref}}\, E_{\text{ref}}}, \\ E_{\text{elec},i} &= P_{\text{base}}\, T + n_{\text{img},i}\,(e_{\text{img}} + e_{\text{tx}}), \qquad E_{\text{ref}} = (P_{\text{base}} + P_{\text{cam}} + P_{\text{tx}})\, T_{\max} \end{aligned} \tag{3.5}$$

ΔV_i is the velocity change needed to reach the orbit from a common circular, equatorial arrival orbit of radius r_0 (Section 4.5); $3v_0$, with $v_0 = \sqrt{\mu/r_0}$, is larger than any such manoeuvre. $n_{\text{img},i}$ is the number of frames spacecraft i takes, each costing the exposure energy e_{img} and the energy e_{tx} to send it to Earth.

Mission time (minimised). The duration enters as $T/T_{\max}$.

3.5 Fitness function

The four objectives are combined into one weighted fitness that both SA and GA minimise:

$$\boxed{J(\mathbf{x}) = w_C\,(1 - C) + w_Z\, Z + w_E\, E + w_T\,\frac{T}{T_{\max}}} \tag{3.6}$$

with the weights $(w_C, w_Z, w_E, w_T) = (0.5,\ 0.15,\ 0.15,\ 0.2)$. Coverage is the only maximised quantity; it enters as $1 - C$, the share of the sunlit area that was missed, so maximising C and minimising $1 - C$ have the same optimum. The weights add to one. For fleets no larger than the reference, the terms are in the usual unit range; for $N > n_{\text{ref}}$, the fixed-reference cost and energy terms can exceed one by design. The ideal limiting interpretation $J = 0$ would be the whole sunlit area imaged instantly at no cost and no energy. The form $1 - C$ is used rather than $1/C$ because $1/C$ is infinite for a fleet that images nothing (common among random designs) and would make the last, hardest percentages of coverage almost worthless. With these weights

one more spacecraft ($w_Z \Delta Z = 0.015$, plus its share of the energy term) pays for itself only if it adds about three to four percentage points of coverage, and shortening the mission by 10 % of $T_{\max}$ is worth four points; this is what stops the optimiser from simply sending every spacecraft for as long as possible.

3.6 Constraints

Every constraint is written as $g \leq 0$ and normalised, so a positive value is the relative size of the violation (Table 3.3).

Table 3.3: Constraints. Index i runs over the flying spacecraft ($s_i = 1$), $t_k = k\,\Delta t$ over the time steps of the mission.

Name	Meaning	Normalised form (≤ 0 when satisfied)	Checked
fleet size	at least one spacecraft, switches binary	$N \geq 1$, $s_i \in \{0, 1\}$	before engine
duration	mission length within its range	$\max(T_{\min} - T,\, T - T_{\max})/T_{\max}$	before engine
g_1 minimum radius	no orbit inside the safety sphere $r_{\min} = \kappa R_{\max}$	$\max_i (r_{\min} - a_i)/r_{\min}$	before engine
g_2 maximum radius	two-body model stays valid, camera stays useful	$\max_i (a_i - r_{\max})/r_{\max}$	before engine
inclination	physical range of ι	$\max_i \max(-\iota_i,\, \iota_i - 180°)/180°$	before engine
g_3 separation	no two spacecraft closer than d_{sep} at any step $t_k \leq T$	$\big(d_{\text{sep}} - \min_{i<j,\,k} \lVert \mathbf{r}_i(t_k) - \mathbf{r}_j(t_k) \rVert\big)/d_{\text{sep}}$	before engine
g_4 battery	state of charge never below $q_{\min} = 20\,\%$ at any step $t_k \leq T$	$(q_{\min} - \min_{i,k}\, q_i(t_k))/q_{\min}$	after engine

For a circular orbit the periapsis and apoapsis both equal a_i, so g_1 and g_2 reduce to bounds on a_i. Ω_i and $M_{0,i}$ are angles: every real value is a valid orbit, so they need no constraint. All constraints except the battery are checked *before* the expensive simulation; a design that fails them is never simulated (Chapter 5). Infeasible designs receive a penalised cost above every feasible J, so the optimiser can still rank them by how badly they fail:

$$J_{\text{inf}}(\mathbf{x}) = J_0 + \rho \sum_j \big[\max(0, g_j(\mathbf{x}))\big]^2, \qquad J_0 = \rho = 10^3. \tag{3.7}$$

3.7 Parameters

The method is independent of the body: Table 3.4 lists the values for both bodies. All Kleopatra distances keep the same proportions to the body as for Eros ($r_{\min} = 1.5 R_{\max}$, $r_{\max} \approx 3.8\, r_{\min}$),

which needs a coarser resolution limit of 20 m per pixel (as Dawn mapped the similar-sized Vesta [7]); Kleopatra's mass is that of Brož et al. [8]. Everything else is identical.

Table 3.4: Mission and problem parameters

Parameter	Symbol	433 Eros	216 Kleopatra
Gravitational parameter	μ	4.463×10^{-4} km^3/s^2	0.1982 km^3/s^2
Rotation period	P_{rot}	5.27 h	5.385 h
Shape model used in the search	—	Gaskell, 1 499 facets[a]	radar, 4 092 facets
Farthest surface point	$R_{\max}$	17.6 km	114.2 km
Surface area	—	1 124 km^2	52 186 km^2
Orbit radius range	$r_{\min}, r_{\max}$	26.5 – 100 km	171.2 – 650 km
Minimum separation	d_{sep}	2 km	10 km
Resolution limit	$\mathrm{GSD}_{\max}$	3 m	20 m
Usable camera range	$\ell_{\max}$	58.6 km	390.8 km
Arrival orbit radius	r_0	50 km	320 km
Camera (all spacecraft)	ψ, p	6° field of view, 2048×2048 pixels	
Maximum emission angle	$\eta_{\max}$	65°	
Lighting	—	Sun $\geq 20°$ above the facet horizon and no cast shadow	
Time step	Δt	10 min (one frame per step, set by the 503 s downlink of a frame)	
Power	—	$P_{\mathrm{base}} = 50$ W, $P_{\mathrm{cam}} = 10$ W, $P_{\mathrm{tx}} = 30$ W, solar 180 W, battery 200 Wh	
Energy per frame	$e_{\mathrm{img}}, e_{\mathrm{tx}}$	100 J, 15.1 kJ	
Minimum battery charge	$q_{\min}$	20 %	
Weights	$w_C, \ldots, w_T$	0.5, 0.15, 0.15, 0.2	
Cost reference	n_{ref}	10 spacecraft	
Penalty	J_0, ρ	10^3, 10^3	

[a] The matched GA–SA comparison of Section 10.7 uses a 3 000-facet copy; every final Eros design is re-checked on a 6 000-facet copy (validation V7).

3.8 Assumptions

- **Two-body dynamics.** Each spacecraft follows an ideal circular Kepler orbit around the asteroid's centre of mass; the irregular gravity field, solar radiation pressure and third bodies are ignored.
- **Uniform rotation** of the asteroid about its inertial z axis.
- **Fixed Sun direction** in inertial space during the mission (sub-solar latitude $0°$, equinox).
- **Nadir pointing.** Every camera points at the asteroid's centre; one frame is taken per time step whenever at least one facet is visible.
- **Discrete time.** Visibility, separation and battery are checked at the time steps t_k only.

- **Facet-level observation.** A facet counts as photographed if its centre passes all the tests of Section 4.4.

Chapter 4

Simulation Model

The fitness J of Chapter 3 is computed by a mission simulation, the *engine*. Given a feasible design it propagates every spacecraft, turns its positions into the rotating asteroid frame, decides which facets it photographs at each step, and tracks its battery. All sizes, speeds and times are kept at their real values (Figure 4.1).

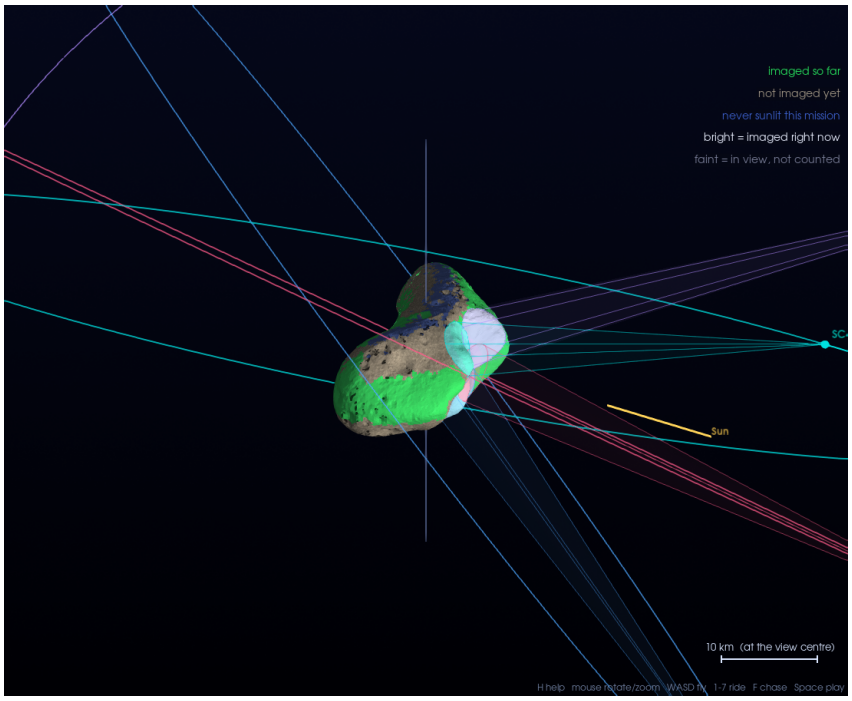


Figure 4.1: The simulator at true scale: the best design of case A2 (four spacecraft, Eros) after 4.4 h, with the orbits and the 6° camera cones; green facets are already photographed, dark blue facets are never sunlit during the mission, the yellow line points to the Sun.

4.1 Asteroid mesh

The surface is a triangle mesh (Figure 4.2). Facet f has area A_f, centre $\mathbf{c}_f$ and outward unit normal $\mathbf{n}_f$, all in the body frame B that spins with the asteroid. Eros uses NASA's Gaskell shape model [9] (49 152 facets, from NEAR Shoemaker images) decimated to 1 500 facets for the search; every final design is re-checked on a 6 000-facet copy (validation V7). Kleopatra uses the radar shape model of Ostro et al. [10] (4 092 facets), shifted to its centre of mass.

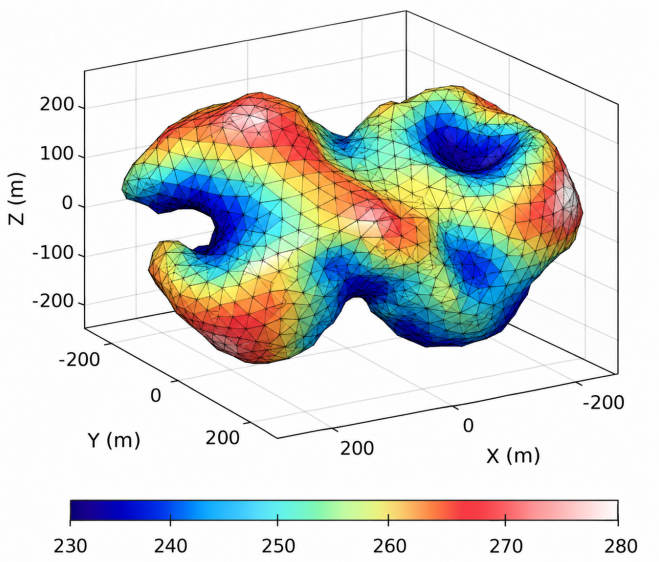


Figure 4.2: The asteroid as a triangle mesh: every facet is one small flat piece of the surface.

4.2 Stage 1: orbit propagation

For a circular orbit of radius a_i the spacecraft moves at the constant mean motion

$$n_i = \sqrt{\mu / a_i^3}, \qquad u_i(t) = M_{0,i} + n_i\, t, \tag{4.1}$$

where u_i is the angle travelled along the orbit from the ascending node. Its inertial position is the in-plane point turned into place by the node and the inclination:

$$\mathbf{r}_i^I(t) = R_3(-\Omega_i)\, R_1(-\iota_i)\, \left[a_i \cos u_i(t),\ a_i \sin u_i(t),\ 0\right]^T, \tag{4.2}$$

with R_1, R_3 the elementary rotations about the x and z axes. (The code propagates general Kepler ellipses with a Newton solver of Kepler's equation; with $e = 0$ it reduces to the above.) For Eros at $a = 35$ km the period is $2\pi/n = 17.1$ h, while Eros turns once every 5.27 h, so the surface slides beneath every orbit.

4.3 Stage 2: rotation into the asteroid frame

Instead of rotating the mesh at every step, each spacecraft position is rotated into the spinning body frame:

$$\mathbf{r}_i^B(t) = R_z(\omega_{\text{ast}} t)^T \, \mathbf{r}_i^I(t), \qquad \omega_{\text{ast}} = 2\pi / P_{\text{rot}}. \tag{4.3}$$

4.4 Stage 3: which facets are photographed

For spacecraft i at step k let $\mathbf{r} = \mathbf{r}_i^B(t_k)$, $\mathbf{q} = \mathbf{r} - \mathbf{c}_f$ (facet to spacecraft), $\ell = \|\mathbf{q}\|$, $\hat{\mathbf{q}} = \mathbf{q}/\ell$ and $\hat{\mathbf{b}} = -\mathbf{r}/\|\mathbf{r}\|$ (nadir boresight). Facet f is photographed with usable quality, $V_{ifk} = 1$, only if all five tests hold:

(a) inside the camera cone: $\hat{\mathbf{b}} \cdot (-\hat{\mathbf{q}}) \geq \cos(\psi/2)$ (4.4)

(b) not too oblique: $\mathbf{n}_f \cdot \hat{\mathbf{q}} \geq \cos \eta_{\max}$ (4.5)

(c) line of sight free: the ray from $\mathbf{c}_f$ towards $\mathbf{r}$ hits no other facet (4.6)

(d) sharp enough: $$\text{GSD} = \frac{2\,\ell \tan(\psi/2)}{p} \leq \text{GSD}_{\max} \iff \ell \leq \ell_{\max} = \frac{\text{GSD}_{\max}\, p}{2 \tan(\psi/2)} \tag{4.7}$$

(e) sunlit: $\mathbf{n}_f \cdot \hat{\mathbf{s}}(t_k) \geq \cos 70^\circ$ and the ray from $\mathbf{c}_f$ (4.8)

towards the Sun hits no other facet (4.9)

$\hat{\mathbf{s}}(t_k)$ is the Sun direction in the body frame. Test (c) and the shadow part of (e) cast rays with the Möller–Trumbore intersection test [11], accelerated with Embree through `trimesh`. The cheap tests (d), (a), (b), (e) are applied first, and only the surviving facet–step pairs are ray-cast. The lighting test (e) does not depend on the design, so it is computed once per case study; it also defines A^* in (3.3).

The observations are then combined over time and across the fleet:

$$v_{if} = \max_{k:\, t_k \leq T} V_{ifk}, \qquad u_f = \max_i v_{if}, \qquad n_{\text{img},i} = \#\{k :\, t_k \leq T,\ \textstyle\sum_f V_{ifk} > 0\}. \tag{4.10}$$

4.5 Stage 4: ΔV, power and battery

ΔV. From the circular equatorial arrival orbit r_0, a spacecraft first tilts its plane, then transfers to radius a_i with a Hohmann transfer:

$$\Delta V_i = 2 v_0 \sin\frac{\iota_i}{2} + \left| \sqrt{\mu\left(\tfrac{2}{r_0} - \tfrac{1}{a_t}\right)} - v_0 \right| + \left| \sqrt{\tfrac{\mu}{a_i}} - \sqrt{\mu\left(\tfrac{2}{a_i} - \tfrac{1}{a_t}\right)} \right|, \qquad a_t = \frac{r_0 + a_i}{2}. \tag{4.11}$$

For Eros $v_0 = 3.0$ m/s, so a polar orbit costs about 4.2 m/s for the plane change alone.

Eclipses and battery. A spacecraft is in eclipse when it is behind the asteroid (a sphere of the body's mean radius) as seen from the Sun. Its battery (200 Wh, starting full) is updated every step:

$$Q_i(t_{k+1}) = \min\Big\{Q_{\mathrm{cap}},\; Q_i(t_k) + \Delta t\,\big[P_{\mathrm{sol}}\,(1-\epsilon_{ik}) - P_{\mathrm{base}} - \delta_{ik}\,\tfrac{e_{\mathrm{img}}+e_{\mathrm{tx}}}{\Delta t}\big]\Big\}, \qquad q_i(t_k) = \frac{Q_i(t_k)}{Q_{\mathrm{cap}}}, \tag{4.12}$$

where $\epsilon_{ik} = 1$ in eclipse and $\delta_{ik} = 1$ when a frame is taken. The lowest q_i enters constraint g_4.

4.6 Making the simulation fast enough

One evaluation simulates N spacecraft over up to $N_t = T_{\max}/\Delta t + 1$ steps (1 441 for ten days) against every facet. Two design decisions keep the cost manageable:

- **Per-spacecraft tracks.** Everything in stages 1–4 depends only on one spacecraft's own orbit. The engine therefore simulates each spacecraft separately over the whole horizon $T_{\max}$ and caches the result (its *track*) under its four orbit variables. A design of duration T simply uses the steps $t_k \le T$ of its tracks. Changing T or switching a slot on or off therefore costs no new simulation, and moving one orbit costs one track.
- **Vectorised geometry.** Tests (a), (b) and (d) need only the products $\mathbf{r}\cdot\mathbf{c}_f$ and $\mathbf{r}\cdot\mathbf{n}_f$, so a block of steps is two matrix products instead of a loop over facets.

Chapter 5

Simulated Annealing Implementation

5.1 Code structure: Algorithm → Feasibility → Engine

The code is organised as a pipeline of three independent layers (Figure 5.1). The *algorithm* only sees a box of numbers and a cost; the *feasibility* layer decides whether a vector describes a mission that may fly; the *engine* simulates feasible missions and returns J. A design that fails a pre-engine check never reaches the (expensive) engine and receives the penalised cost (3.7) directly. Because the algorithm only relies on a small `Problem` interface (bounds, variable kinds, `repair`, `evaluate`), the same SA code also runs on standard benchmark functions (validation V2), and further algorithms plug into the same pipeline through a registry: the GA of this milestone (Chapter 9) and PSO later.

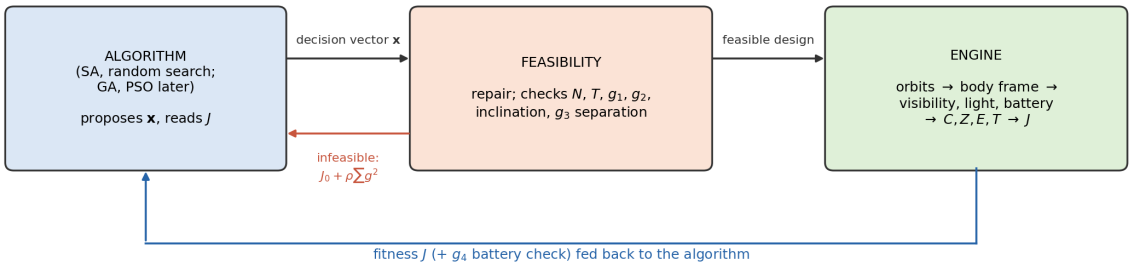


Figure 5.1: The optimisation pipeline. The algorithm proposes a decision vector; the feasibility layer rejects (with a penalty) or passes it; the engine simulates the mission and returns the fitness, which is fed back to the algorithm.

The SA loop is written from scratch with NumPy only; no optimisation toolbox or package is used anywhere in the algorithm.

Table 5.1: Main code modules (package `spacecov`)

Module	Content
`config.py`	all mission, constraint and cost parameters; body presets (Eros, Kleopatra)
`design.py`	decision vector layout, bounds, variable kinds, encode/decode (Eq. 3.1)
`feasibility.py`	constraints of Table 3.3, the `repair` operator
`physics.py`	stages 1–4 of Chapter 4 for one spacecraft
`engine.py`	track cache, fleet simulation, objectives and J (Eq. 3.6)
`pipeline.py`	Algorithm → Feasibility → Engine; the `Problem` the algorithms see
`algorithms/sa.py`	Simulated Annealing (this chapter)
`algorithms/ga.py`	Genetic Algorithm (Chapter 9)
`algorithms/random_search.py`	random search with the same budget (baseline of validation V4)
`algorithms/benchmarks.py`	sphere, Rastrigin and Ackley test functions (validation V2)
`experiments/validate.py`	every case study and validation run of Chapters 6–8
`viz/`	interactive 3D viewer and live optimiser display

5.2 The algorithm

SA [12] keeps one current design and repeatedly proposes a small change. A better design is always accepted; a worse one is accepted with the Metropolis probability [13] $\exp(-\Delta J/\Theta)$, which is high while the temperature Θ is high and vanishes as it cools. Early on the search can therefore leave regions that are only locally good; at the end it fine-tunes the best region. (The temperature is written Θ here to avoid confusion with the mission duration T.) Algorithm 1 gives the implementation and Figure 5.2 the corresponding flowchart.

5.3 Problem-specific considerations

Table 5.2 summarises how the generic algorithm is adapted to the fleet problem.

Neighbourhood operator. A move changes one or two coordinates (chosen with equal probability), drawn only among the coordinates that currently *matter*: the duration T, every switch s_i, and the four orbit variables of the *flying* spacecraft. The orbit numbers of a switched-off slot are never moved, because changing them would not change the mission and would waste an evaluation. A real coordinate j receives Gaussian noise $\mathcal{N}\big(0, (\sigma\,(\mathrm{ub}_j - \mathrm{lb}_j))^2\big)$, so the step is a fraction σ of the variable's range. A switch is an integer, so it is flipped (the noise is rounded to at least ± 1 and the repair clips it back to 0 or 1): this adds or removes a spacecraft, which is how SA searches the fleet size. One move therefore changes at most one or two spacecraft, and thanks to the track cache (Chapter 4) it costs at most two new spacecraft simulations whatever the fleet size.

Repair operator. After a move the vector is mapped back into the box: Ω_i and $M_{0,i}$ wrap around $360°$; a_i, ι_i and T are *reflected* at their bounds (a step that would leave $[r_{\min}, r_{\max}]$ by 3 km lands 3 km inside), which keeps the random walk unbiased near the bounds; switches are

Algorithm 1 Simulated Annealing for the fleet design

Require: budget B evaluations, $n_{\text{init}} = 20$, $p_0 = 0.5$, $\sigma_0 = 0.25$, $\sigma_{\min} = 0.01$, $\Theta_{\min}/\Theta_0 = 10^{-3}$

Warm start: draw n_{init} random designs that pass the pre-engine checks, evaluate them, $\mathbf{x} \leftarrow$ the best

Start temperature: make 20 trial moves from $\mathbf{x}$; $\Theta_0 \leftarrow -\operatorname{median}(\Delta J^{+})/\ln p_0$ over the feasible uphill moves

$L \leftarrow \max(10, \dim \mathbf{x})$; $\quad n_{\text{lev}} \leftarrow \lceil (B - \text{evaluations used})/L \rceil$; $\quad \alpha \leftarrow (\Theta_{\min}/\Theta_0)^{1/(n_{\text{lev}}-1)}$

$\Theta \leftarrow \Theta_0$; $\quad \mathbf{x}^{*} \leftarrow \mathbf{x}$

while the budget is not used **do**

  for L moves **do**

    $\sigma \leftarrow \max\bigl(\sigma_{\min},\ \sigma_0\sqrt{\Theta/\Theta_0}\bigr)$ (moves shrink as it cools)

    $\mathbf{y} \leftarrow \textsc{Repair}(\textsc{Neighbour}(\mathbf{x}, \sigma))$

    $J(\mathbf{y}) \leftarrow$ Feasibility $\rightarrow$ Engine; $\quad \Delta J \leftarrow J(\mathbf{y}) - J(\mathbf{x})$

    if $\Delta J \le 0$ **or** $\operatorname{rand}() < \exp(-\Delta J/\Theta)$ **then**

      $\mathbf{x} \leftarrow \mathbf{y}$ (Metropolis acceptance)

    end if

    if $\mathbf{y}$ is better than $\mathbf{x}^{*}$ (feasible first, then lower J) **then**

      $\mathbf{x}^{*} \leftarrow \mathbf{y}$

    end if

  end for

  $\Theta \leftarrow \alpha\,\Theta$ (geometric cooling)

end while

return $\mathbf{x}^{*}$

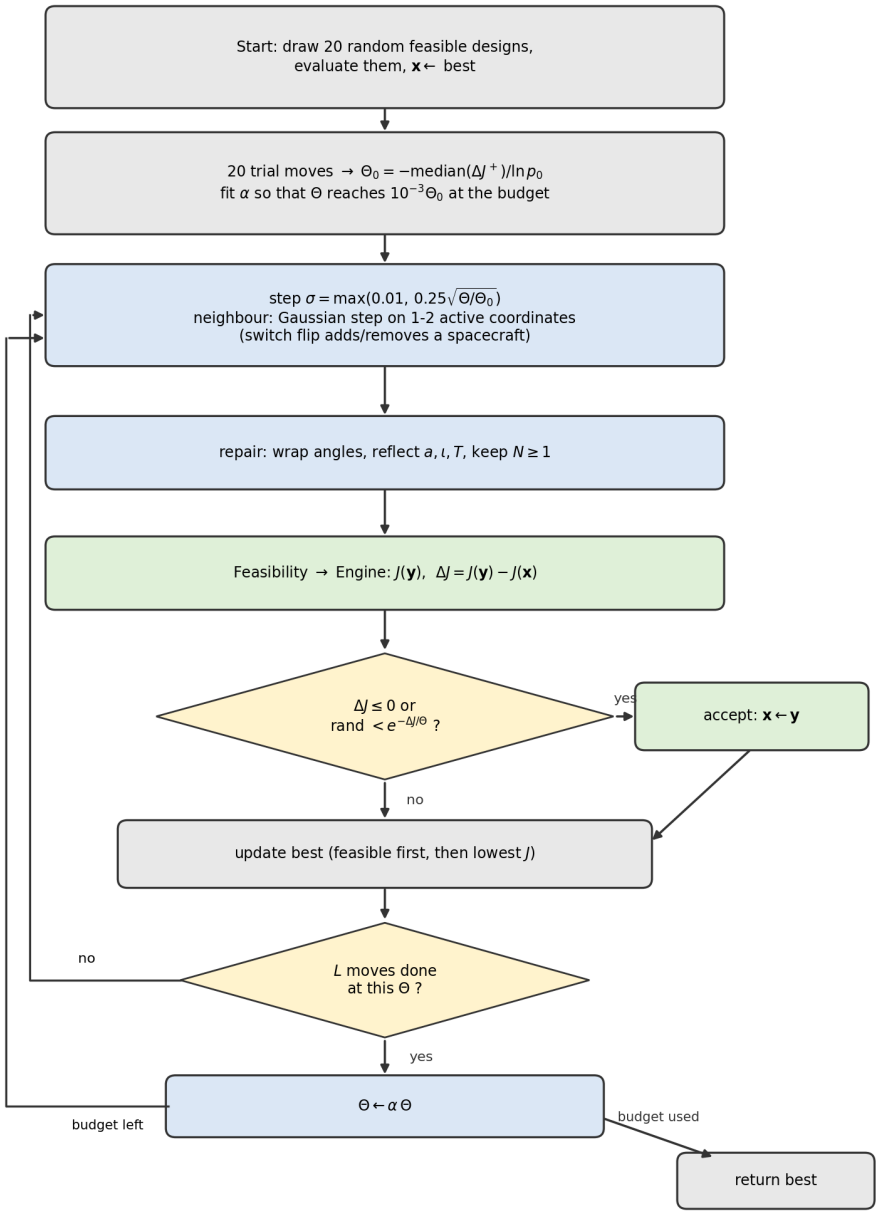


Figure 5.2: Flowchart of the Simulated Annealing implementation.

rounded to 0 or 1. If a move would switch off the last spacecraft, the repair switches it back on, so $N \geq 1$ always holds. Constraints g_1, g_2, the duration range and the inclination range can therefore never be violated by an SA candidate; only separation (g_3) and battery (g_4) can still fail. A move that the repair maps back onto the current design, or widens to more than two coordinates, is redrawn, so every move stays local.

Warm start. The narrow camera only works up to $\ell_{\max} = 58.6\,\mathrm{km}$ (Eros), so a large part of the box (high orbits, short missions) photographs almost nothing: J is high and flat there. A start temperature measured in such a region is far too hot for the region of good designs and the budget is wasted on a random walk. Starting from the best of 20 random designs (counted in the budget) places the search, and the measurement of Θ_0, where the moves matter.

Start temperature. $\Theta_0 = -\operatorname{median}(\Delta J^+)/\ln p_0$ makes a typical uphill move accepted with probability $p_0 = 0.5$ at the start. It adapts automatically to the scale of J in every case study; the median ignores the rare penalty spikes of infeasible trial moves.

Cooling schedule. The temperature is lowered geometrically, $\Theta \leftarrow \alpha\Theta$, after every Markov chain of $L = \max(10, \dim \mathbf{x})$ moves, that is 26 moves for $N_{\max} = 5$ and 51 for $N_{\max} = 10$. The factor α is fitted so that the temperature reaches $\Theta_{\min} = 10^{-3}\Theta_0$ exactly when the budget runs out (for 2000 evaluations and $N_{\max} = 5$, $\alpha \approx 0.91$). The move size shrinks with $\sqrt{\Theta/\Theta_0}$ from 25 % to 1 % of each range, from exploration to fine-tuning.

Feasible-first best. The reported design is the best *feasible* design ever evaluated; among feasible designs the one with the lowest J. A design that breaks a constraint can be the current point of the walk (it is accepted only rarely because of its penalty) but is never reported.

5.4 Algorithm parameters

Table 5.3 lists the default parameters, used in every case study unless stated otherwise. Their effect is studied in Section 8.5.

5.5 Visualisation of the search and of the solutions

The interactive viewer (package `viz`, PyVista and Qt) shows any design in 3D at true scale: the asteroid with its coverage colouring, the orbits, the camera cones, the Sun direction, and a timeline that replays the mission step by step (Figure 5.3). The optimiser can be run from the viewer: the 3D scene follows the design SA is currently on, every proposed move is explained (which spacecraft, which variable, ΔJ, acceptance probability), and live charts show the coverage over time, the fitness, every design tried and the temperature. The same scene code renders the figures of Chapter 8.

Table 5.2: Problem-specific ingredients of the SA implementation

Ingredient	What we do	Why
Neighbourhood	Gaussian step on 1–2 active coordinates; switches flip	local moves; N is searched through the switches
Active coordinates	only T, the switches and orbits of flying spacecraft	no evaluation is wasted on unused slots
Repair	wrap angles, reflect a, ι, T, round switches, keep $N \geq 1$	g_1, g_2, T and ι ranges always hold
Warm start	best of 20 random feasible designs	avoids the flat region beyond camera range
Start temperature	$\Theta_0 = -\operatorname{median}(\Delta J^+)/\ln p_0$, $p_0 = 0.5$	adapts to the scale of J
Cooling	geometric, α fitted to the budget, $L = \dim \mathbf{x}$	the whole budget is used, from exploration to fine-tuning
Step size	$\sigma = \max(0.01,\ 0.25\sqrt{\Theta/\Theta_0})$	large moves early, small moves late
Constraint handling	pre-engine rejection + quadratic penalty	infeasible designs cost no simulation and are ranked
Best design	feasible first, then lowest J	the reported fleet never breaks a constraint
Evaluation cost	per-spacecraft track cache	a move costs at most two spacecraft simulations

Table 5.3: Default SA parameters

Parameter	Symbol	Default
Evaluation budget (per run)	B	2000
Random designs for the warm start	n_{init}	20
Trial moves to estimate Θ_0	—	20
Initial acceptance probability	p_0	0.5
Final temperature	$\Theta_{\min}/\Theta_0$	10^{-3}
Chain length (moves per temperature)	L	$\max(10, \dim \mathbf{x})$
Cooling factor	α	fitted to B (≈ 0.91 for $N_{\max} = 5$)
Initial / smallest step (fraction of range)	σ_0 / $\sigma_{\min}$	0.25 / 0.01
Coordinates changed per move	—	1 or 2

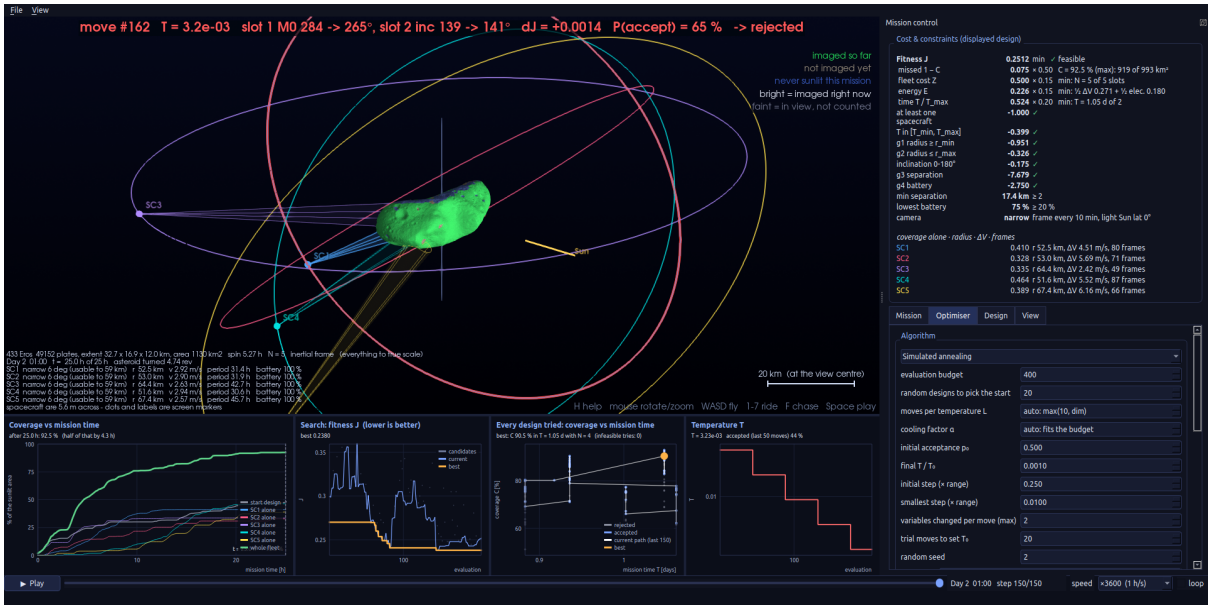


Figure 5.3: The viewer while SA runs: 3D scene with the current fleet (left), objectives, constraints and optimiser settings (right), live charts of coverage, fitness, designs tried and temperature (bottom).

Chapter 6

Case Studies

The algorithm is tested on case studies of increasing size and difficulty. They differ in the body (Eros, 1 499 facets; Kleopatra, 46 times the surface and 4 092 facets), the longest allowed mission $T_{\max}$ (from two hours to ten days, i.e. from 13 to 1 441 simulated steps), and the number of slots $N_{\max}$ (5 or 10, i.e. 26 or 51 decision variables). Short missions are the hard ones: the asteroid turns only a few times, so *where* each spacecraft is *when* decides what can be seen, and the fitness landscape becomes rugged.

In every case the mission duration is searched in $[T_{\min}, T_{\max}]$ with $T_{\min} = \min(6\,\mathrm{h},\, T_{\max}/4)$, and all other settings are those of Tables 3.4 and 5.3. Every SA run uses 2000 evaluations, and every case is repeated with independent random seeds (a seed fixes all random choices of one run: start designs, moves and acceptances), so that the mean and spread of the results can be reported. Table 6.1 lists the cases and their input parameters.

Cases A and B are also solved by random search with the same budget (validation V4). Cases C1–C3 fix every switch to 1 (so $N = N_{\max}$) and optimise only the orbits and the duration; each is compared with a free-N SA run of the same mission with $N_{\max} = 10$. In case E one parameter at a time is changed from the defaults of Table 5.3 (values in Section 8.5).

Table 6.2 shows how the cost of one run grows with the case. The time per evaluation depends mostly on how many new spacecraft tracks have to be simulated and on their length ($T_{\max}/\Delta t$ steps) times the number of facets.

Table 6.1: Case studies and their input parameters (budget 2000 evaluations per run)

Case	Body, setting	$T_{\max}$	$T_{\min}$	$N_{\max}$	dim	Seeds	Question
A1	Eros	10 d	6 h	5	26	5	long mission: coverage saturates
A2	Eros	1 d	6 h	5	26	5	How do fleet size, orbits and duration change as the time available shrinks?
A3	Eros	1 d	6 h	10	51	5	
A4	Eros	12 h	3 h	10	51	5	
A5	Eros	6 h	1.5 h	10	51	5	
A6	Eros	2 h	0.5 h	10	51	5	
B1	Kleopatra	3 d	6 h	5	26	5	Does the method carry over unchanged to a 6.5× larger, strongly concave body?
B2	Kleopatra	3 d	6 h	10	51	5	
B3	Kleopatra	12 h	3 h	10	51	5	
C1	Eros, N forced to 1–6	2 d	6 h	N	$1{+}5N$	5	Is the fleet size SA chooses really the best? (V6)
C2	Eros, N forced to 1–8	6 h	1.5 h	N	$1{+}5N$	5	
C3	Kleopatra, N forced to 1–5	3 d	6 h	N	$1{+}5N$	3	
D	Eros, no Sun / shadows	10 d	6 h	5	26	5	comparison with the literature (V5)
E	Eros, SA parameters varied	2 d	6 h	5	26	5	effect of α, p_0, σ_0, warm start

Table 6.2: Size of the case studies and the cost of one SA run (2000 evaluations, one CPU core; nine single-threaded runs in parallel on one machine)

Setting	Body	$T_{\max}$	$N_{\max}$	Variables	Steps N_t	Minutes per run	Evaluations / s
Eros 10 d	Eros	10 d	5	26	1441	2.6	12.8
Eros 1 d	Eros	1 d	5	26	145	0.3	99.9
Eros 1 d	Eros	1 d	10	51	145	0.3	109.0
Eros 12 h	Eros	12 h	10	51	73	0.2	176.7
Eros 6 h	Eros	6 h	10	51	37	0.2	191.7
Eros 2 h	Eros	2 h	10	51	13	0.1	349.5
Kleopatra 3 d	Kleopatra	3 d	5	26	433	2.2	15.4
Kleopatra 3 d	Kleopatra	3 d	10	51	433	2.1	16.1
Kleopatra 12 h	Kleopatra	12 h	10	51	73	0.4	83.5

Chapter 7

Validation

SA always returns *some* design, so its results are only meaningful if every link of the chain is checked: the simulator must be correct, the algorithm must find optima that are known, the problem must leave something to optimise, the algorithm must beat blind sampling, its results must agree with the literature, its choice of fleet size must be the right one, and its designs must not be artefacts of the coarse model. Table 7.1 lists the seven checks and the rest of the chapter reports them. All numbers are produced by `experiments/validate.py`; its full log, every run's best design and convergence curve, and the run settings are saved in `results/validation/`. The objective uses a fixed reference fleet of ten spacecraft, so fleets larger than ten remain penalised consistently across all cases.

Table 7.1: Validation plan

	Check	Question answered
V1	physics tests	Does the simulator agree with hand calculations?
V2	known optima	Does SA find the known minimum of standard test functions?
V3	saturation	Do random fleets already cover everything (nothing to optimise)?
V4	SA vs random search	Is SA significantly better than blind sampling with the same budget?
V5	literature comparison	Under the assumptions of published work, do we reach the same coverage?
V6	forced fleet size	Is the number of spacecraft SA chooses the best one?
V7	finer mesh	Do the designs keep their coverage on a four times finer asteroid model?

7.1 V1: the simulator obeys physics

An automated test suite (90 tests, run with every change of the code) checks the simulator against hand calculations (Table 7.2) and checks the pipeline itself: decoding and encoding of the decision vector, every constraint and the repair operator, the penalty, the fixed-N mode, the cost terms, and the bookkeeping of the algorithms.

Table 7.2: V1: physics checks (selection)

Check	Expected	Simulator
Kepler solver residual, $e \leq 0.5$	$< 10^{-10}$	$< 10^{-10}$
Circular orbit radius along the orbit	constant	constant
Orbital period at $a = 35$ km (Eros)	$2\pi\sqrt{a^3/\mu} = 17.1$ h	17.1 h
Energy and angular momentum along an orbit	constant	constant to 0.1 %
ΔV of a $50 \rightarrow 30$ km transfer	Hohmann formula	identical
Usable range of the 6° camera at 3 m/pixel	58.6 km	58.6 km
Facet hidden behind terrain (concave test mesh)	not visible	not visible
Facet behind a blocking plate, Sun behind the plate	shadowed	shadowed
Eclipse behind the asteroid, battery drain	hand-computed cases	identical

7.2 V2: SA finds known optima

Because the SA code is generic, it can be run unchanged on standard test functions whose global minimum is known ($f = 0$ at $\mathbf{x} = \mathbf{0}$): the smooth sphere function in five dimensions, and the highly multi-modal Rastrigin function and the Ackley function, each in two and five dimensions (Table 7.3, 10 runs of 10 000 evaluations each, default parameters).

Table 7.3: V2: SA on test functions with known minimum 0 (10 runs of 10 000 evaluations)

Function	Dim.	Best	Mean	Worst	Runs with $f < 0.1$
Sphere	5	1.2e-03	2.5e-03	4.0e-03	100 %
Rastrigin	2	7.8e-05	2.1e-02	1.2e-01	90 %
Rastrigin	5	6.8e-03	4.7e-01	2.0e+00	70 %
Ackley	2	2.8e-03	1.2e-02	2.4e-02	100 %
Ackley	5	9.2e-03	1.9e-02	3.5e-02	100 %

SA reaches the global minimum of the smooth functions in every run: the sphere function in five dimensions and the Ackley function in two and five. The Rastrigin function has a regular grid of local minima around the global one: in two dimensions 9 of 10 runs reach it, in five dimensions 7 of 10, and the other runs end in a neighbouring local minimum (mean 0.47, i.e. about half a unit above the optimum). This is the expected behaviour of a single-point method with a finite budget on a deceptive landscape, and it shows that the algorithm is implemented correctly: it converges, it escapes local minima most of the time, and it does not stall.

7.3 V3: is there anything to optimise?

If random fleets already photographed almost the whole surface, any optimiser would look good. Fifty random designs of one, three and five spacecraft were therefore flown for ten days around Eros (Figure 7.1, Table 7.4).

Random fleets are far from saturating the surface. A single random spacecraft images 38 %

Table 7.4: V3: coverage of random designs flown for ten days around Eros

N	Designs	Mean C	Min	Max	Share with $C \geq 95\,\%$
1	50	38 %	0 %	77 %	0 %
3	50	71 %	0 %	96 %	2 %
5	50	88 %	69 %	97 %	14 %

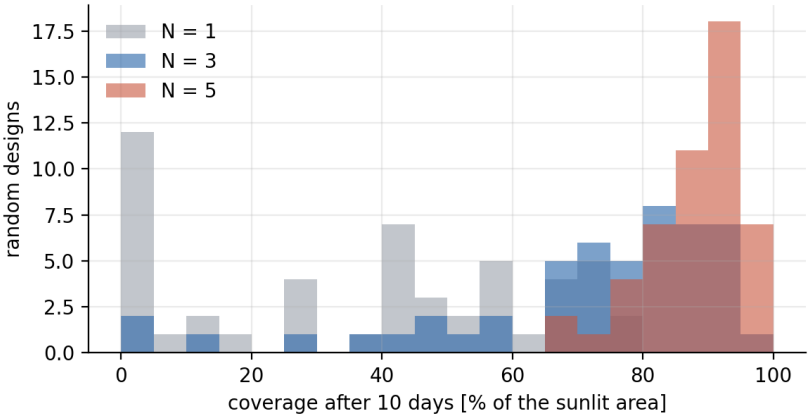


Figure 7.1: V3: coverage reached by random designs after ten days on Eros.

of the sunlit area on average, three image 71 %, and even five spacecraft flying for the full ten days image 88 %. Only 2 % of the three-spacecraft designs and 14 % of the five-spacecraft designs exceed 95 %, and these random designs also pay the full ten days and the cost of every spacecraft. Good designs are therefore rare and the problem is worth optimising. The spread (from 0 to 96 % for three spacecraft) also shows how strongly the orbits matter: an orbit beyond the camera range images nothing.

7.4 V4: SA against random search

Random search draws designs uniformly in the same box (with the same repair) and keeps the best feasible one; with the same number of evaluations it is the minimum any optimiser must beat. Both methods were run five times in each of the nine settings of cases A and B. Whether SA's fitness is lower is tested with the one-sided Mann–Whitney U test [14], which makes no assumption about the distribution of the results; with five runs per method the smallest possible p-value is $1/\binom{10}{5} = 0.004$, reached when every SA run beats every random-search run.

SA wins in all nine settings and in every single run: in each setting the worst of the five SA runs is better than the best of the five random-search runs, so the Mann–Whitney test gives the smallest possible $p = 0.004$ nine times. The gain comes mostly from coverage, from $+6$ points in the long ten-day Eros mission (95.1 against 88.8 %) to $+21$ points in the two-hour mission

Table 7.5: V4: SA (bold) against random search with the same budget; means over 5 runs each

Setting	$N_{\max}$	$J \downarrow$	C [%] $\uparrow$	N	T [d]	worst SA / best RS J	p
Eros 10 d	5	**0.108** / 0.167	**95.1** / 88.8	**2.2** / 2.8	**2.03** / 2.95	0.117 / 0.159	0.004
Eros 1 d	5	**0.212** / 0.309	**88.6** / 69.7	**3.0** / 3.0	**0.45** / 0.47	0.226 / 0.285	0.004
Eros 1 d ($N_{\max}$ 10)	10	**0.205** / 0.313	**89.6** / 74.1	**3.2** / 3.4	**0.43** / 0.55	0.221 / 0.287	0.004
Eros 12 h	10	**0.277** / 0.404	**80.5** / 67.8	**3.4** / 4.4	**0.26** / 0.36	0.294 / 0.371	0.004
Eros 6 h	10	**0.355** / 0.475	**75.0** / 55.0	**3.4** / 4.4	**0.19** / 0.19	0.398 / 0.444	0.004
Eros 2 h	10	**0.443** / 0.538	**38.7** / 17.3	**2.8** / 2.2	**0.03** / 0.03	0.492 / 0.524	0.004
Kleopatra 3 d	5	**0.225** / 0.310	**74.8** / 64.3	**2.6** / 2.8	**0.69** / 1.10	0.241 / 0.300	0.004
Kleopatra 3 d ($N_{\max}$ 10)	10	**0.231** / 0.316	**76.3** / 66.2	**2.8** / 4.4	**0.86** / 0.89	0.254 / 0.303	0.004
Kleopatra 12 h	10	**0.340** / 0.460	**61.6** / 47.0	**2.8** / 4.4	**0.22** / 0.26	0.362 / 0.456	0.004

p: one-sided Mann–Whitney test that SA's J is lower. "worst SA / best RS": if the first number is lower, every SA run beat every random-search run.

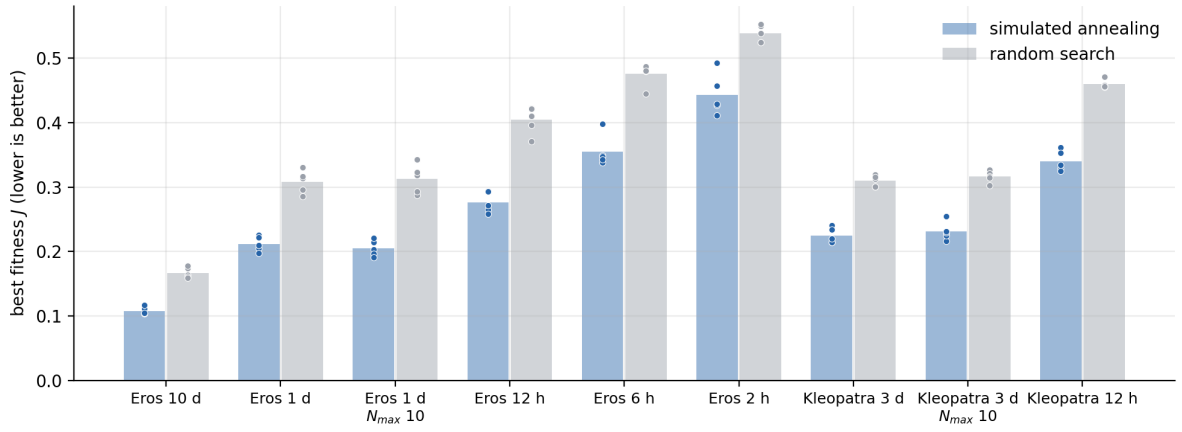


Figure 7.2: V4: best fitness of SA and of random search with the same budget (bars: means, dots: single runs; lower is better).

(38.7 against 17.3 %). In the short missions random search also needs more spacecraft (mean 4.4 against 3.4 at 12 h and 6 h on Eros) and still sees less, and in the long missions it needs longer missions (mean 2.9 against 2.0 days for $T_{\max} = 10$ d). The advantage is smallest when the mission is long, because the asteroid then turns many times under every orbit and almost any reasonable fleet ends up seeing most of the surface (V3); it is largest when time is short, because *where* each spacecraft is *when* decides what can be seen, and random sampling does not find such combinations. Raising $N_{\max}$ from 5 to 10 (51 instead of 26 variables) does not hurt SA: its results in cases A2/A3 and B1/B2 differ by less than one standard deviation.

7.5 V5: comparison with the literature

The closest prior work is by Nallapu and Thangavelautham [3], who designed a swarm that maps Eros during a single flyby: a genetic algorithm chose the swarm size and the positions on a ring, and 8 spacecraft gave 100 % coverage at 1 m resolution. Their model assumes the whole surface is lit and only checks whether a facet faces the camera. In follow-up work that counts only the sunlit side they mapped 99.1 % of Mars' moon Deimos [2]. Stolfi Rosso and Danoy [6] tuned swarm formations around an asteroid with a genetic algorithm and found SA the more accurate method for on-board tuning. To compare under the same assumptions, case D switches the Sun and shadow tests off (every facet counts as lit) and solves the ten-day Eros mission again.

Table 7.6: V5: comparison with the closest prior work on Eros

Work	Trajectories	Lighting	Spacecraft	Coverage (resolution)	Algorithm
Nallapu & Thangavelautham [3]	one flyby (10 min)	ignored	8	100 % (1 m)	GA
This work, case D	orbits, $T \leq 10$ d	ignored	2, 2, 2, 2, 2	97.9 % (3 m)	SA
This work, case A1	orbits, $T \leq 10$ d	Sun + shadows	3, 2, 2, 2, 2	95.1 % (3 m)	SA

With the Sun and shadow tests switched off, SA images 97.9–98.1 % of the *whole* Eros surface in every run, with two spacecraft and a mission of about 1.2 days. This is close to the 100 % of Nallapu and Thangavelautham, so under the same assumptions our model and optimiser give the same answer. The remaining two percent are deep concavities that the steep 65° emission limit and the occlusion test, which the reference does not apply, make invisible from any orbit. The difference in fleet size (2 against 8) is the expected trade-off between a fast flyby, which has to surround the body at once, and orbiting for a day, which lets the spin of Eros bring the surface to the cameras. With realistic lighting (case A1) coverage drops to 95.1 % of the sunlit area, so the lower coverage of our main results comes from the stricter lighting model, not from the optimiser.

7.6 V6: is the chosen fleet size the best?

In cases C1–C3 the fleet size is forced to $N = 1, 2, 3, \ldots$ and only the orbits and the duration are optimised. If SA's own choice in the free problem is right, the best forced N should coincide with it (Figure 7.3, Table 7.7).

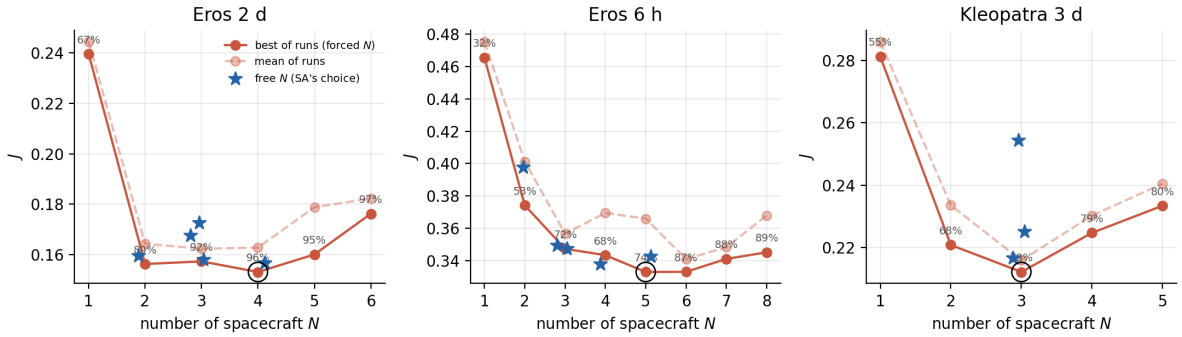


Figure 7.3: V6: best fitness when the fleet size is forced to N (line) and the free-N SA runs (stars, at the N they chose).

Table 7.7: V6: forced fleet size against the fleet size SA chooses itself

Mission	N	Runs	Best J	Mean J	Mean C	Mean T	N chosen
Eros 2 d	1	5	0.2395	0.2444	67.4 %	14.3 h	
Eros 2 d	2	5	0.1564	0.1644	89.5 %	17.0 h	
Eros 2 d	3	5	0.1574	0.1624	92.5 %	15.3 h	
Eros 2 d	4	5	0.1532	0.1629	95.9 %	15.6 h	
Eros 2 d	5	5	0.1601	0.1788	94.7 %	12.3 h	
Eros 2 d	6	5	0.1763	0.1823	96.9 %	12.6 h	
Eros 2 d	free ($N_{\max}$ 10)	5	0.1567	0.1630	92.4 %	15.4 h	3, 2, 3, 3, 4
Eros 6 h	1	5	0.4653	0.4753	31.6 %	3.3 h	
Eros 6 h	2	5	0.3742	0.4013	53.1 %	3.6 h	
Eros 6 h	3	5	0.3474	0.3568	71.6 %	4.4 h	
Eros 6 h	4	5	0.3435	0.3696	68.3 %	3.8 h	
Eros 6 h	5	5	0.3331	0.3660	73.5 %	3.7 h	
Eros 6 h	6	5	0.3332	0.3411	86.6 %	4.1 h	
Eros 6 h	7	5	0.3412	0.3484	87.9 %	4.0 h	
Eros 6 h	8	5	0.3453	0.3679	88.5 %	4.0 h	
Eros 6 h	free ($N_{\max}$ 10)	5	0.3382	0.3552	75.0 %	4.5 h	3, 4, 2, 3, 5
Kleopatra 3 d	1	3	0.2813	0.2861	54.8 %	14.3 h	
Kleopatra 3 d	2	3	0.2208	0.2335	68.2 %	13.7 h	
Kleopatra 3 d	3	3	0.2121	0.2158	78.6 %	17.6 h	
Kleopatra 3 d	4	3	0.2247	0.2302	79.1 %	15.8 h	
Kleopatra 3 d	5	3	0.2334	0.2405	79.6 %	14.8 h	
Kleopatra 3 d	free ($N_{\max}$ 10)	3	0.2167	0.2321	77.2 %	21.0 h	3, 3, 3

Forcing the fleet size shows a clear picture: one spacecraft is always far worse, the fitness is lowest for a moderate fleet, and large fleets become worse again because their cost outweighs the few extra points of coverage. Near the optimum, however, the fitness is very *flat*: for the two-day Eros mission $N = 2, 3, 4$ lie within 0.005 of each other, and for the six-hour mission $N = 5$ and $N = 6$ are equal and $N = 3$–8 lie within 0.015. Inside this flat region the choice

of N is almost a tie, and the free-N SA runs land in it: for Kleopatra every run chose $N = 3$, exactly the best forced size; for the two-day Eros mission the runs chose 2–4 spacecraft and the best of them ($J = 0.1567$) is within 0.004 of the best forced design; for the six-hour mission they chose 2–5 spacecraft (best $J = 0.3382$, within 0.006 of the best forced $N = 5$). The free runs tend to choose one or two spacecraft fewer than the forced optimum. Adding a spacecraft is a single move whose immediate effect is often negative (its orbit is still untuned, but it already costs $\Delta Z = 0.1$), so near the end of the run, when the temperature is low, such moves are rarely accepted. The forced runs do not have to take this step. This is a known weakness of a single-point method on a variable-size problem; a population-based method that keeps several fleet sizes alive may handle it better (the GA of Chapter 9 starts from a population stratified over the fleet size).

7.7 V7: finer asteroid model

The search uses a 1 499-facet copy of the Eros model for speed. Every best Eros design of cases A, C, D and E, and of the random-search runs of V4, was re-simulated on a 6 000-facet copy (200 designs, 30 of them from random search).

On the finer model the coverage of the 200 re-checked designs is on average 2.23 percentage points lower (at most 4.48), and J is 0.011 higher on average. The coarse mesh is therefore slightly optimistic, because its larger facets count as photographed as soon as their centre is, but the shift is small and nearly the same for every design (it does not change which designs are good). For the final numbers of a real mission the best designs should be re-evaluated on the fine model, as is done here.

7.8 Summary of the validation

All seven checks pass. The simulator agrees with hand calculations (V1); SA finds known optima (V2); the problem leaves a lot to optimise (V3); SA beats random search with the same budget in every setting and every run, with $p = 0.004$ (V4); under the assumptions of the literature it reproduces the near-complete coverage reported there (V5); its fleet sizes lie in the flat optimum found by forcing N, exactly at the optimum for Kleopatra and within 0.006 for Eros, with a slight tendency to send too few spacecraft (V6); and its designs keep their coverage within a few points on a four times finer model (V7). All 248 final designs (193 validation runs and 55 runs of the parameter study) are feasible.

Chapter 8

Results and Discussion

This chapter reports the designs SA found in every case study (Section 8.1), shows them at work (Section 8.2), analyses what the optimiser learned (Section 8.3) and then analyses the algorithm itself: convergence, cost and repeatability (Section 8.4) and the effect of its parameters (Section 8.5).

8.1 Fleets designed for every case study

Table 8.1 lists the results of all SA runs. Every final design is feasible.

Table 8.1: Fleets designed by SA in every case study (mean over runs; N is listed per run)

Case	Body	$T_{\max}$	$N_{\max}$	N per run	T	J (mean $\pm$ std)	C	Z	E	$T/T_{\max}$	Feas.
A1	Eros	10 d	5	3, 2, 2, 2, 2	2.03 d	0.108 ± 0.006	95.1 %	0.22	0.066	0.20	5/5
A2	Eros	1 d	5	3, 2, 3, 3, 4	10.9 h	0.212 ± 0.012	88.6 %	0.30	0.129	0.45	5/5
A3	Eros	1 d	10	3, 4, 4, 3, 2	10.3 h	0.205 ± 0.013	89.6 %	0.32	0.128	0.43	5/5
A4	Eros	12 h	10	4, 4, 3, 2, 4	6.3 h	0.277 ± 0.016	80.5 %	0.34	0.157	0.52	5/5
A5	Eros	6 h	10	3, 4, 2, 3, 5	4.5 h	0.355 ± 0.024	75.0 %	0.34	0.192	0.75	5/5
A6	Eros	2 h	10	2, 4, 4, 1, 3	0.8 h	0.443 ± 0.032	38.7 %	0.28	0.120	0.38	5/5
B1	Kleopatra	3 d	5	3, 3, 3, 2, 2	16.6 h	0.225 ± 0.012	74.8 %	0.26	0.091	0.23	5/5
B2	Kleopatra	3 d	10	3, 3, 3, 3, 2	20.7 h	0.231 ± 0.014	76.3 %	0.28	0.090	0.29	5/5
B3	Kleopatra	12 h	10	2, 3, 3, 3, 3	5.2 h	0.340 ± 0.016	61.6 %	0.28	0.127	0.44	5/5
D	Eros	10 d	5	2, 2, 2, 2, 2	1.16 d	0.073 ± 0.004	97.9 %	0.20	0.063	0.12	5/5

Less time, more spacecraft, and only the time that pays. Figure 8.1 shows how the designs change with the longest allowed mission on Eros. With ten days available, SA sends two spacecraft (four runs of five) and *ends the mission after about two days* (44–55 h), by which time they have imaged 95 % of the sunlit area: the asteroid turns about nine times under every orbit in two days, after which the coverage gained per extra day is worth less than its cost in time (w_T). When less time is available the surface no longer comes to the spacecraft, so more spacecraft are sent: two to four at one day and twelve hours, two to five at six hours, and the

mission uses a larger share of the time allowed (20 % of $T_{\max}$ for ten days, 45 % for one day, 75 % for six hours). Coverage falls from 95 % (ten days) to 89 % (one day), 81 % (twelve hours) and 75 % (six hours). At two hours Eros turns by only a third of a revolution and each spacecraft takes at most a dozen frames, so coverage collapses to 39 % and the runs disagree (1 to 4 spacecraft); here SA even stops after 0.5–1 h, because the last frames add too little. SA never fills all the slots it is allowed: with $N_{\max} = 10$ it sends the same 2–4 spacecraft as with $N_{\max} = 5$, because each spacecraft has to pay for itself through coverage.

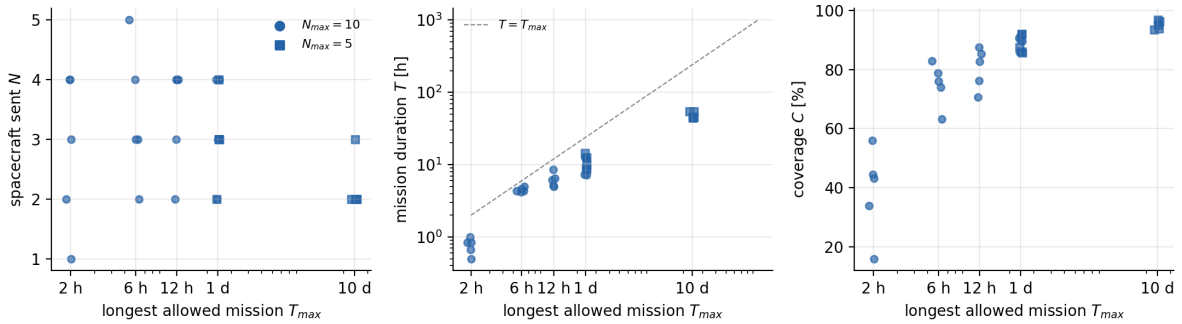


Figure 8.1: Eros: fleet size, mission duration and coverage chosen by SA against the longest allowed mission $T_{\max}$ (each marker is one run).

Second body. The same code and settings, applied to Kleopatra (46 times the surface of Eros), send two or three spacecraft for 12–33 h when three days are allowed and image 75–76 % of the sunlit area; with twelve hours they send two or three spacecraft for about five hours and image 62 %. Coverage is lower than for Eros with a comparable mission length because Kleopatra is much more concave (it fills only 64 % of its convex hull against 84 % for Eros): the narrow waist between its two lobes hides part of the surface from most viewing directions (Figure 8.4).

8.2 The best designs at work

Figures 8.2–8.4 replay the best designs of three cases in the viewer (on the 6 000-facet Eros model for display; green: photographed so far, grey: not yet, blue: never sunlit during the mission).

The replays show how the fleets work. In the ten-day case (Figure 8.2) the two spacecraft see about a third of the sunlit area in the first rotation of Eros; the rest is collected as the spin brings new longitudes under the orbits, and the mission is stopped once the remaining facets come in too slowly to pay for more time. In the six-hour case (Figure 8.3) four spacecraft are spread around the body in different planes so that, together, they see both ends of Eros within about one rotation. On Kleopatra (Figure 8.4) two spacecraft image three quarters of the sunlit

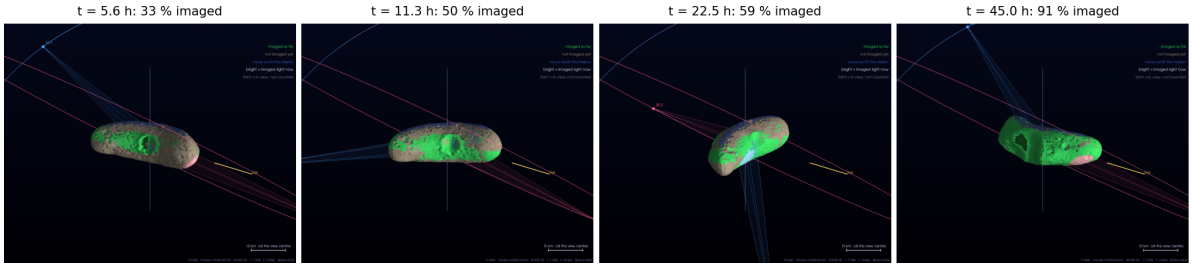


Figure 8.2: Case A1 (Eros, $T_{\max} = 10\,\mathrm{d}$): the best design at four moments of its mission.

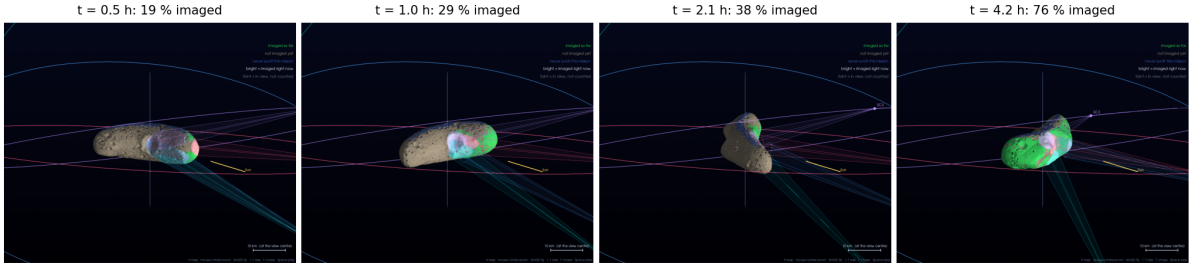


Figure 8.3: Case A5 (Eros, $T_{\max} = 6\,\mathrm{h}$): the best design at four moments of its mission.

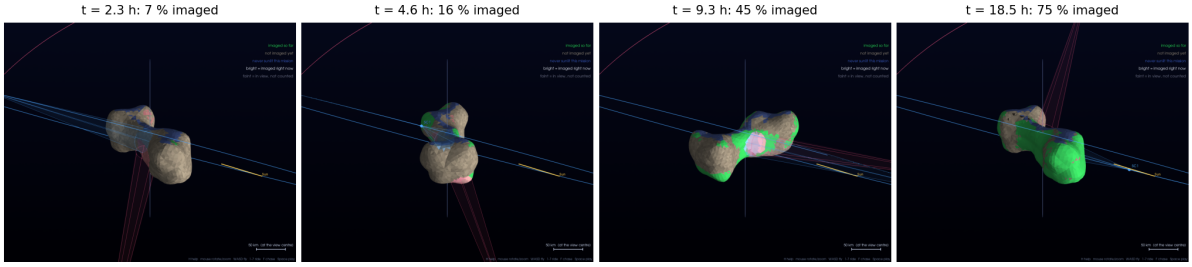


Figure 8.4: Case B1 (Kleopatra, $T_{\max} = 3\,\mathrm{d}$): the best design at four moments of its mission.

area in 18.5 h. The coverage values in the figure titles are computed on the 6 000-facet display model, so they are a few points lower than those of Table 8.1 (validation V7).

8.3 What the optimiser learned

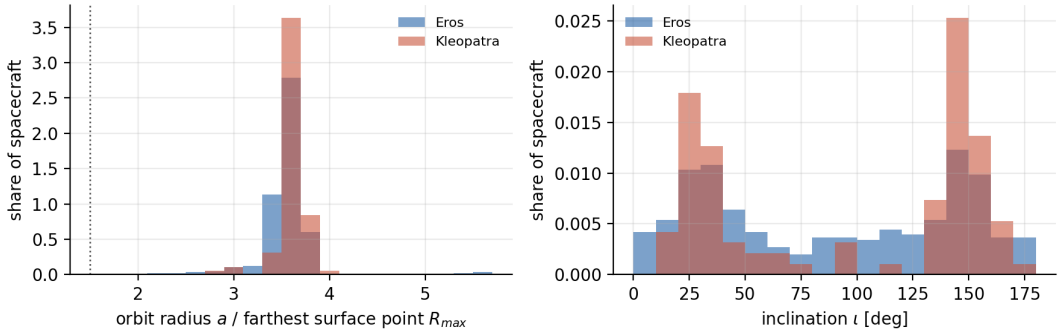


Figure 8.5: Orbits of every spacecraft in the best feasible SA designs: radius in units of the farthest surface point (dotted line: $r_{\min} = 1.5R_{\max}$) and inclination.

Figure 8.5 collects every orbit of the best designs and reveals two consistent rules.

- **The same height in proportion to the body.** The orbit radii cluster tightly at 3.5–3.6 times the farthest surface point for *both* bodies (median 62.4 km for Eros, 409 km for Kleopatra; 79 % and 88 % of the orbits lie within ±5 % of the median). This is just above the usable camera range (58.6 km and 391 km): the optimiser flies as high as it can while the near side of the body is still within range, which maximises the footprint of every frame. Nothing in the code tells it this; it emerges from the simulation, and it scales correctly from a 34 km to a 219 km body.
- **Tilted, not polar, orbits.** The median tilt of the orbits from the equator is 35° for Eros and 31° for Kleopatra. With the Sun in the equatorial plane, facets near the poles never see the Sun 20° above their horizon and are not part of A^*, so a polar orbit would spend time over regions that do not count; a moderate tilt, combined with the spin and the 65° emission limit, sweeps the whole lit band. Prograde and retrograde orbits are about equally common (50 % and 57 % retrograde): the extra plane-change ΔV of a retrograde orbit changes J by only about 0.003, so the fitness hardly distinguishes them.
- **Batteries are never the limit; separation sometimes is.** Re-evaluating the 45 best designs of cases A and B, the lowest battery charge is 58 % (limit 20 %) and the median is 100 %: the 180 W panels more than cover the 50 W bus plus imaging. Separation, in contrast, is close to active in three short-mission Eros designs (closest approach 2.7–3.0 km against the 2 km limit), where several spacecraft share the space just above the camera range.

8.4 Algorithm performance

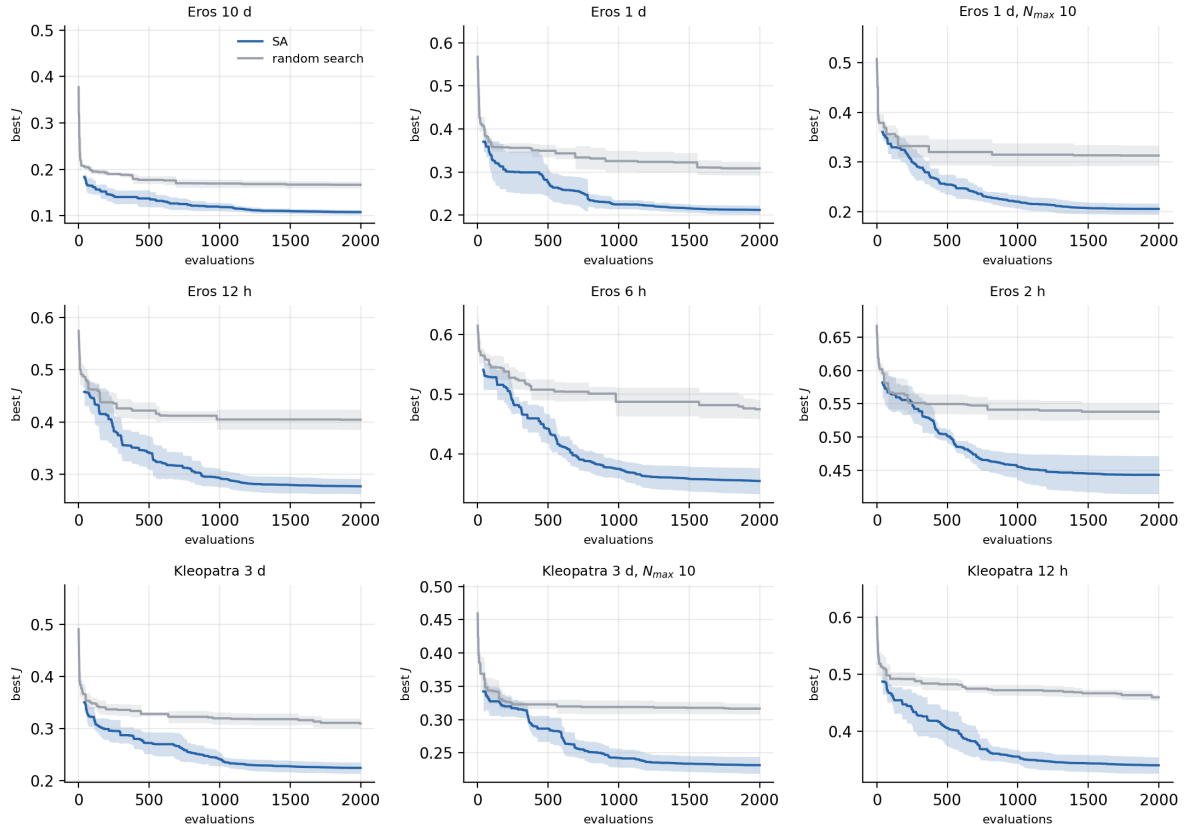


Figure 8.6: Convergence: best fitness against evaluations, mean ± one standard deviation over 5 runs, for SA and random search in every case of A and B.

Convergence. Figure 8.6 shows that SA improves quickly during the first 500–1000 evaluations, while the temperature is high and large moves are accepted, and then fine-tunes the orbits; by 1500 evaluations the curves are nearly flat, so the budget of 2000 is adequate. Random search improves only in rare jumps and stalls early, because the chance of drawing a better design at random falls quickly. The warm start is visible as SA's curves starting at evaluation 40, already at a level random search reaches only after a hundred or more draws.

Repeatability. The standard deviation of the final J over five seeds is 0.006–0.016 in most cases and 0.024–0.032 for the hardest, six- and two-hour Eros missions (Table 8.1). Different seeds therefore give designs of very similar quality, but not identical designs: the fleet size in particular differs between runs where the fitness is flat in N (validation V6).

Computational cost. Table 6.2 gives the time of one run on one CPU core. It grows with the number of time steps and facets, from a few seconds for a two-hour Eros mission to about 2–3 minutes for ten days on Eros or three days on Kleopatra. It hardly depends on $N_{\max}$, because a move changes at most two spacecraft and the unchanged ones are taken from the track cache.

The same cache explains why random search, which simulates every spacecraft of every new design from scratch, simulates 2.5 to 6 times more spacecraft tracks and is 1.9 to 4.4 times slower than SA for the same number of evaluations (e.g. 5.8 against 2.6 minutes for ten days on Eros). The whole validation (193 runs) took about one hour on six cores.

8.5 Effect of the algorithm parameters

Case E changes one parameter at a time from the defaults (Table 5.3) on the two-day Eros mission ($N_{\max} = 5$), with five runs per setting: the cooling factor α (fixed at 0.80, 0.95 or 0.98 instead of fitted to the budget, in which case a run stops when $\Theta < 10^{-3}\Theta_0$ even if budget is left), the initial acceptance probability p_0 (0.3, 0.8, 0.95), the initial step σ_0 (0.05, 0.10, 0.50) and the warm start (one random start instead of the best of 20).

Table 8.2: Case E: effect of the SA parameters (Eros, $T_{\max} = 2\,\mathrm{d}$, $N_{\max} = 5$; one parameter changed at a time from the defaults)

Setting	α used	Runs	J (mean $\pm$ std)	Best	Worst	Mean C	Mean N	Evaluations
default	0.912	5	0.1602 ± 0.0044	0.1540	0.1645	92.3 %	2.4	2000
$\alpha = 0.80$	0.800	5	0.1930 ± 0.0267	0.1591	0.2273	92.2 %	2.6	846
$\alpha = 0.95$	0.950	5	0.1674 ± 0.0027	0.1635	0.1702	90.3 %	2.4	2000
$\alpha = 0.98$	0.980	5	0.2082 ± 0.0155	0.1889	0.2225	85.6 %	2.8	2000
$p_0 = 0.30$	0.912	5	0.1626 ± 0.0110	0.1503	0.1781	91.2 %	2.6	2000
$p_0 = 0.80$	0.912	5	0.1720 ± 0.0136	0.1635	0.1956	89.9 %	2.6	2000
$p_0 = 0.95$	0.912	5	0.2065 ± 0.0275	0.1824	0.2508	87.6 %	2.6	2000
$\sigma_0 = 0.05$	0.912	5	0.1883 ± 0.0219	0.1607	0.2215	88.2 %	2.4	2000
$\sigma_0 = 0.10$	0.912	5	0.1887 ± 0.0407	0.1552	0.2538	88.7 %	2.2	2000
$\sigma_0 = 0.50$	0.912	5	0.1600 ± 0.0074	0.1521	0.1714	92.3 %	2.8	2000
one random start	0.913	5	0.1593 ± 0.0055	0.1532	0.1674	92.0 %	2.4	2000

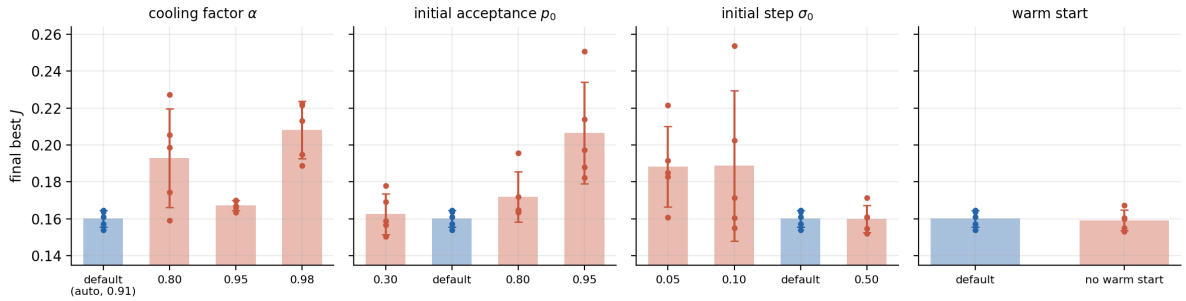


Figure 8.7: Case E: final best fitness for each parameter value (bars: means, dots: runs; blue: the default).

Cooling factor α: the most important parameter. The default, α fitted to the budget (0.912 here), gives the best and most consistent result ($J = 0.160 \pm 0.004$). A fixed $\alpha = 0.80$ cools so fast that the temperature reaches $10^{-3}\Theta_0$ after only 846 evaluations: the run ends early, frozen in whatever region it reached, and the results scatter widely (0.193 ± 0.027). The slow schedules

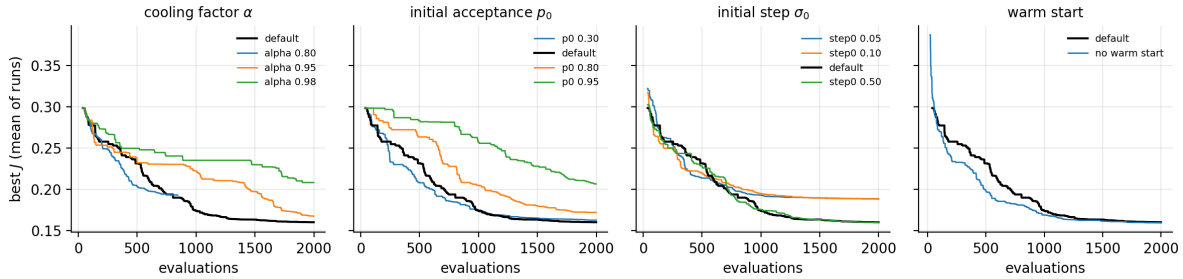


Figure 8.8: Case E: mean convergence for each parameter value.

waste the budget on the opposite side: with $\alpha = 0.95$ the temperature at the end of the budget is still 2 % of Θ_0 and with $\alpha = 0.98$ still 22 %, so the search never settles to fine-tune ($J = 0.167$ and 0.208). Fitting α to the budget avoids both failures automatically.

Initial acceptance p_0: do not start too hot. $p_0 = 0.3$ and 0.5 give the same quality; $p_0 = 0.8$ and especially 0.95 are clearly worse (0.172 and 0.207). A hot start accepts almost every worsening move for the first part of the run (the flat green and orange curves of Figure 8.8), so a large part of the budget is spent on a random walk.

Initial step σ_0: start with large moves. $\sigma_0 = 0.25$ (default) and 0.5 are equally good (0.160), while small initial steps (0.05, 0.10) are worse and much less reliable (0.188 ± 0.022 and 0.189 ± 0.041): with small steps SA cannot move an orbit far enough to leave a poor region, so the outcome depends on where it started.

Warm start: no effect here. Starting from one random design instead of the best of 20 gives the same result (0.159 against 0.160) in this case. The 20 start evaluations are a small part of the budget, and the adaptive Θ_0 and the cooling recover from a poor start when the mission is two days long and most orbits see something. The warm start was introduced for the flat regions of short missions and high orbits; in this case it is neutral and costs only 1 % of the budget.

With five runs per setting the differences above about 0.01 in mean J are consistent with the spread of the runs; smaller differences (for instance between $p_0 = 0.3$ and 0.5, or $\sigma_0 = 0.25$ and 0.5) should be read as ties.

8.6 Recommendations

- **Let the cooling follow the budget.** Fit α so that the temperature reaches its final value when the budget is used; a fixed α is either too fast (the run stops early) or too slow (it never fine-tunes).
- **Start moderately warm and with large steps**: $p_0 \approx 0.3$–0.5 and $\sigma_0 \approx 0.25$–0.5 of each variable's range.

- **Budget.** About 1500–2000 evaluations are enough for the problem sizes studied; most of the gain comes in the first 1000.
- **Fleet size.** The fitness is flat in N near the optimum and SA tends to send one or two spacecraft too few; for a final design, run SA with the fleet size forced to the chosen N and its neighbours (as in V6), or use a population-based method that keeps several fleet sizes alive.
- **Final numbers.** Re-evaluate the chosen designs on the fine shape model, which lowers the coverage by about two points.

Chapter 9

Genetic Algorithm Implementation

A GA [15, 16] evolves a *population* of candidate designs instead of a single one. In every generation the designs are scored, the better ones are selected as parents, their genes are recombined (crossover) and slightly perturbed (mutation), and the children form the next generation. The GA plugs into the same Algorithm $\rightarrow$ Feasibility $\rightarrow$ Engine pipeline as SA (Figure 5.1): it only proposes decision vectors and reads back their cost, under the same evaluation budget B, so the two algorithms are compared on equal terms. Like SA, it is written from scratch with NumPy only.

9.1 Chromosome representation

The GA uses the decision vector $\mathbf{x}$ of Eq. (3.1) directly as its chromosome; no re-encoding is needed. Four levels are distinguished (Figure 9.1):

- **Gene:** one entry of $\mathbf{x}$, e.g. the orbit radius a_2 of slot 2.
- **Slot** $\mathbf{y}_i = [a_i, \iota_i, \Omega_i, M_{0,i}, s_i]$: the five genes of one spacecraft (its circular orbit and its on/off switch).
- **Chromosome** $\mathbf{x} = [T, \mathbf{y}_1, \ldots, \mathbf{y}_{N_{\max}}]$: one *individual*, i.e. one complete mission design (the whole fleet and the mission duration), with $\dim \mathbf{x} = 1+5N_{\max}$ genes (26 for $N_{\max} = 5$, 51 for $N_{\max} = 10$).
- **Population:** the P chromosomes (candidate missions) of one generation.

Why a chromosome is a whole mission and not one spacecraft. The fitness J (Eq. 3.6) is only defined for a complete design: the coverage C is the area imaged by the *union* of all spacecraft (two spacecraft imaging the same region are worth less than two imaging different regions), the fleet cost Z depends on how many spacecraft fly, the separation constraint g3

couples *pairs* of spacecraft, and the duration T is shared by the whole fleet. The unit that is evaluated and selected must therefore be the full mission.

Why a fixed length. The fleet size is not stored; it follows from the switches, $N = \sum_i s_i$ (Eq. 3.2). Every chromosome has the same $1 + 5N_{\max}$ genes whatever its N, so standard crossover and mutation apply between any two parents, including parents with different fleet sizes, and the GA searches exactly the same space as SA.

Why real coding. The genes are kept as real numbers (and a binary switch) instead of being converted to bit strings: a binary code would add Hamming cliffs (neighbouring values that differ in many bits), a fixed precision, and would break the periodicity of the angles Ω_i and $M_{0,i}$. The existing `repair` operator already handles every gene kind: T, a_i and ι_i are reflected at their bounds, Ω_i and $M_{0,i}$ wrap at 360°, and the switches are rounded to $0/1$ with at least one switched on.

Active and dormant genes. The orbit genes of a switched-off slot are carried along but do not affect J (*dormant* genes). At any time the *active* genes are T, all $N_{\max}$ switches and the four orbit genes of each flying spacecraft, $1 + N_{\max} + 4N$ in total (18 of the 26 genes in Figure 9.1). Mutation acts on active genes only, so no evaluation is spent on a change that cannot alter the fitness; the dormant genes are still inherited, so switching a slot back on can revive an orbit found earlier in the search.

9.2 Population size

The population size P is set from the length of the chromosome:

$$P = \min\Big(60,\ \max\Big(30,\ 10\Big\lceil \tfrac{1.5\,\dim \mathbf{x}}{10} \Big\rceil\Big)\Big) \quad \Rightarrow \quad P = 40\ (N_{\max} = 5,\ \dim \mathbf{x} = 26), \qquad P = 60\ (N_{\max} = 10 \tag{9.1}$$

With elitism (the n_e best individuals are copied unchanged into the next generation), each generation after the first costs $P - n_e$ evaluations, so a budget of B evaluations gives

$$G = 1 + \left\lceil \frac{B - P}{P - n_e} \right\rceil \tag{9.2}$$

generations. Table 9.1 lists the resulting settings. The choice rests on four arguments.

1. Diversity across the genes. A population that is small compared with the number of genes cannot hold enough different values of every gene at once; crossover then has little to recombine and the population collapses onto one design early (premature convergence). A

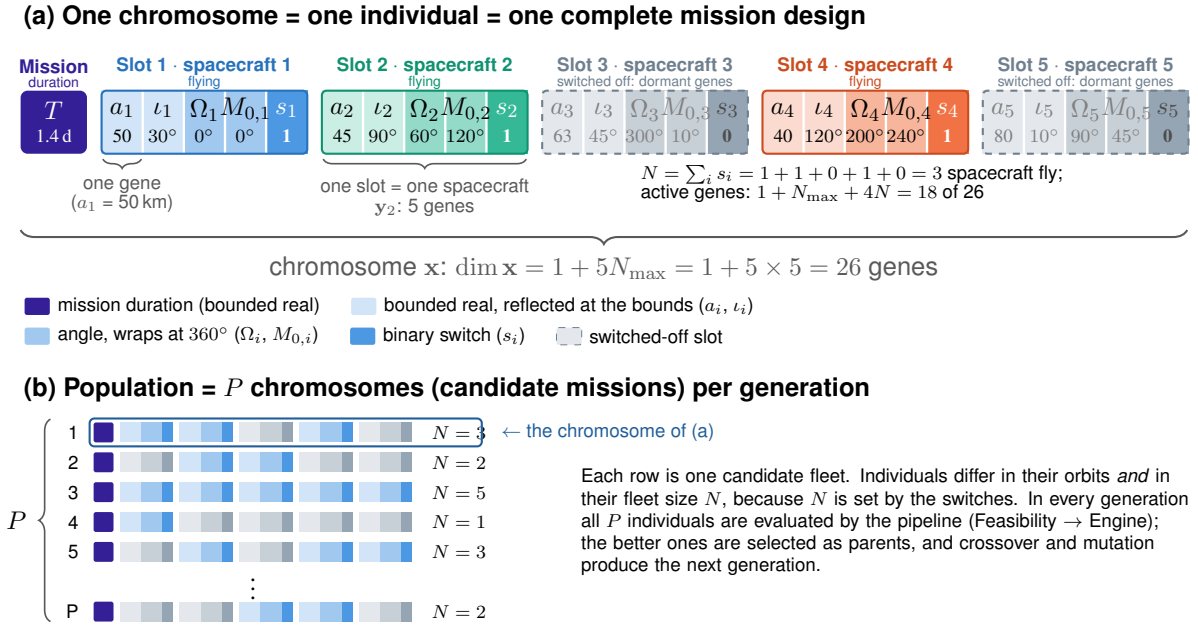


Figure 9.1: Chromosome representation of the GA. (a) A gene is one number, a slot is the five genes of one spacecraft, and the chromosome is the whole decision vector (Eq. 3.1): the mission duration followed by $N_{\max}$ slots. In this example ($N_{\max} = 5$) slots 1, 2 and 4 fly ($N = 3$); the orbit genes of slots 3 and 5 are dormant but still inherited. (b) The population holds P such chromosomes, each a complete mission with its own fleet size.

Table 9.1: Population size and number of generations for the budget $B = 2000$ used by SA, with the default elite share $\varepsilon = 5\,\%$ (Section 9.5).

Case studies	$N_{\max}$	$\dim \mathbf{x}$	P	n_e	G
Eros 10 d / 1 d, Kleopatra 3 d	5	26	40	2	53
Eros 1 d / 12 h / 6 h / 2 h, Kleopatra 3 d / 12 h ($N_{\max} = 10$)	10	51	60	3	36

common rule of thumb is a population between one and two times the number of decision variables; $P \approx 1.5 \dim \mathbf{x}$ sits in the middle of that range.

2. Every fleet size must be represented. The fleet size is encoded by the switches. In the random initial population each switch is on with probability $\frac{1}{2}$, so N follows a binomial distribution (an all-off draw is repaired to $N = 1$). Table 9.2 gives the expected number of individuals per fleet size. With $P = 40$ and $N_{\max} = 5$ every fleet size from 1 to 4 starts with at least six individuals, so the competition between fleet sizes is decided by selection and not by chance. For $N_{\max} = 10$ small fleets are rare in a uniform start (fewer than one individual with $N = 1$, about three with $N = 2$), although SA found that two to four spacecraft are best (Chapter 8); the initial population is therefore spread evenly over the fleet sizes (Section 9.3).

Table 9.2: Expected number of individuals per fleet size N in a uniformly random initial population, $P \cdot \Pr(N)$ with $N \sim \mathrm{Binomial}(N_{\max}, \frac{1}{2})$ and an all-off draw counted as $N = 1$.

	fleet size N									
	1	2	3	4	5	6	7	8	9	10
$N_{\max} = 5$, $P = 40$	7.5	12.5	12.5	6.3	1.3	–	–	–	–	–
$N_{\max} = 10$, $P = 60$	0.6	2.6	7.0	12.3	14.8	12.3	7.0	2.6	0.6	0.1

3. A fixed budget forces a trade-off. GA and SA receive the same budget B, so $P \cdot G \approx B$: every individual added to the population removes evaluations from later generations. A small population converges early; a large one leaves too few generations for selection and crossover to act. In the limit $P = B$ the GA is a single random generation, i.e. random search, which validation V4 already showed to be clearly worse than SA. The cap of 60 for $N_{\max} = 10$ follows from this: $1.5 \times 51 \approx 77$ individuals would leave only about 28 generations.

4. The cost of a generation. The engine caches the simulation of every spacecraft by its four orbit genes (Chapter 5). A child that inherits whole slots from its parents reuses their cached simulations, so the cost of an evaluation comes mainly from mutated orbits, not from the population size. A population holds at most $P\,N_{\max}$ distinct orbits (200 for $N_{\max} = 5$, 600 for $N_{\max} = 10$); the cache size is raised from 256 to 1024 orbits so that the parents' simulations are not evicted while their children are evaluated. Caching does not change any fitness value, only the run time.

The value of Eq. (9.1) is a starting point. It is checked in the parameter study by repeating the runs with $P = 10$, 20, 40, 80 and 120 at the same budget.

9.3 Initial population

Two ways of drawing the first generation are compared:

- **Uniform:** every gene is drawn uniformly within its bounds and every switch is a coin flip. The fleet size then follows a binomial distribution (Table 9.2): medium fleets are common, small and large fleets rare.
- **Stratified** (default): the population is first divided into $N_{\max}$ equal groups, one per fleet size $N = 1, \ldots, N_{\max}$ (the remainder of $P/N_{\max}$, if any, is assigned to randomly chosen sizes). In the group of size N, exactly N randomly chosen slots are switched on; all orbit genes and T are then drawn uniformly as before.

Table 9.3: Number of individuals per fleet size N in the first generation (uniform: expected value).

		fleet size N									
		1	2	3	4	5	6	7	8	9	10
$N_{\max} = 5$, $P = 40$	uniform	7.5	12.5	12.5	6.3	1.3	–	–	–	–	–
	stratified	8	8	8	8	8	–	–	–	–	–
$N_{\max} = 10$, $P = 60$	uniform	0.6	2.6	7.0	12.3	14.8	12.3	7.0	2.6	0.6	0.1
	stratified	6	6	6	6	6	6	6	6	6	6

Why stratified. The fleet size is the most important single decision of the design: it sets the fleet cost Z directly and the coverage through the number of cameras. A uniform start already prefers some sizes before any design has been evaluated: with $N_{\max} = 10$ it contains on average 0.6 designs with one spacecraft and 2.6 with two, but 14.8 with five, although SA found two to four spacecraft to be best. The GA would first have to reduce the fleet size step by step through switch flips. A stratified start gives every fleet size the same number of candidates, so that the comparison between fleet sizes is made by selection, on the fitness, and not by the sampling. It costs nothing: the orbits are drawn exactly as before.

Feasibility of the start. Every individual is drawn until it passes the pre-engine checks (radius band, duration and separation), at most 200 times, as SA's warm start does. A redraw keeps the fleet size of its group, so rejections cannot bias the start towards small fleets; they only reject orbits that come too close. Rejections are also rare: in a test on 400 random designs per fleet size (Eros), at least 93 % pass for every fleet size up to ten spacecraft when $T_{\max} \leq 2$ d, and at least 97 % up to five spacecraft for $T_{\max} = 10$ d; only nine or ten spacecraft over 10 d, with more time to drift close, fall to about 82 %.

Both start types are compared in the parameter study.

9.4 Selection

Parents are chosen by tournament selection: k individuals are drawn at random from the whole population (with replacement) and the best of them, compared feasible first and then by lower J, becomes a parent. The default is the binary tournament, $k = 2$, with the selection probabilities of Eq. (9.4). For crossover, the second parent is drawn again until it is a different individual from the first, since crossing a design with itself only produces a copy.

Tournament selection is used instead of fitness-proportional (roulette-wheel) selection because of the scale of the cost: feasible designs normally have an objective of order one, while a design that fails a constraint costs at least 10^3 (Eq. 3.7); fleets above the reference size may also have $J > 1$. Roulette selection needs positive fitness values on a meaningful scale and would have to transform this cost; a tournament only compares two designs, so it is unaffected by the scale and applies the feasibility-first rule of SA exactly. Its selection pressure is set by k alone: a larger k favours the best designs more strongly and reduces the diversity faster; $k = 2$, 3 and 5 are compared in the parameter study.

9.5 Population split: elites, crossover and mutation

Each new generation is assembled from three groups (Figure 9.2):

$$n_e = \mathrm{round}(\varepsilon P), \qquad n_c = \mathrm{round}\big(\chi\,(P - n_e)\big), \qquad n_m = P - n_e - n_c, \tag{9.3}$$

where ε is the *elite share* and χ the *crossover fraction*. The defaults are $\varepsilon = 5\,\%$ and $\chi = 0.8$ (the same structure as the elite count and crossover fraction of MATLAB's `ga`); Table 9.4 gives the resulting group sizes.

- **Elites** (n_e): the n_e best *distinct* individuals (ranked feasible first, then by lower J; an infeasible design costs more than any feasible one, Eq. 3.7) are copied unchanged. Their fitness is already known, so they are not evaluated again.
- **Crossover children** (n_c): two parents are chosen by tournament, their slots are aligned, and a child is assembled from the slots of both (Section 9.6). If n_c is odd, the last pair contributes one child. A child identical to a member of the population (e.g. one of its parents) is mutated, so that no evaluation is spent on a copy.
- **Mutation children** (n_m): one parent is chosen by tournament, copied, and one or two of its active genes are changed: a Gaussian step on T, a_i, ι_i, Ω_i or $M_{0,i}$, or a flip of a switch s_i; the result is passed through `repair`.

Each generation therefore costs $n_c + n_m = P - n_e$ evaluations, as used in Eq. (9.2).

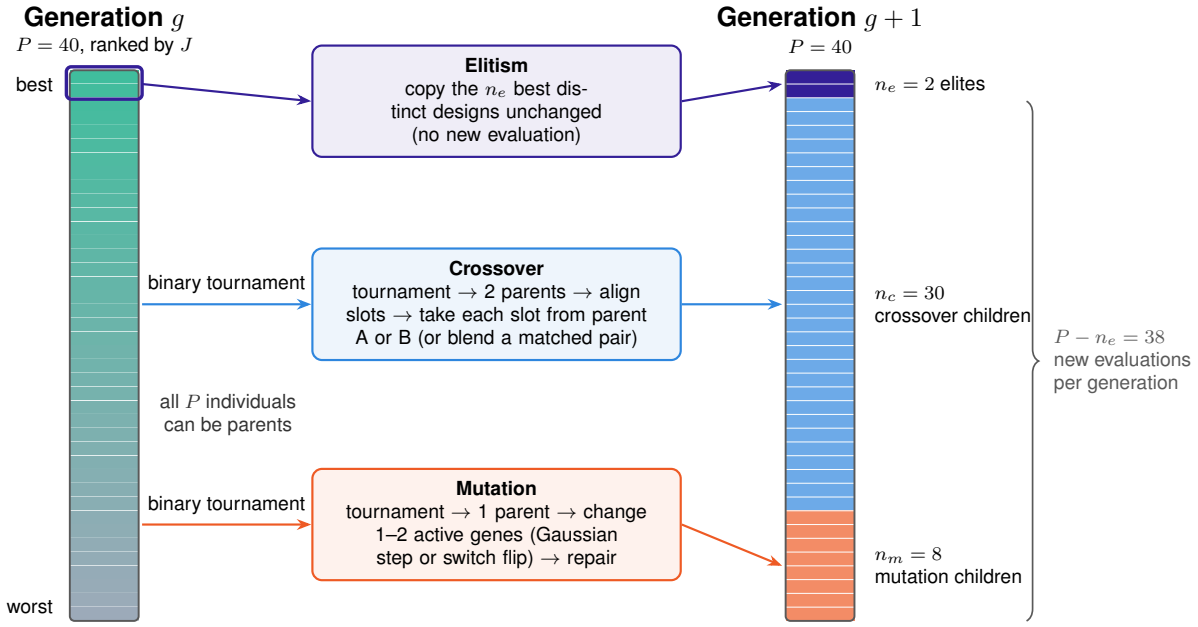


Figure 9.2: Assembly of a new generation ($P = 40$, defaults $\varepsilon = 5\,\%$, $\chi = 0.8$). The two best designs survive unchanged; every other place is filled by a new child, bred from parents chosen by binary tournament among *all* P individuals.

Table 9.4: Group sizes of Eq. (9.3) for the default $\varepsilon = 5\,\%$, $\chi = 0.8$, and the number of generations G (Eq. 9.2) for $B = 2000$.

	P	elites n_e	crossover n_c	mutation n_m	evaluations / generation	G
$N_{\max} = 5$	40	2	30	8	38	53
$N_{\max} = 10$	60	3	46	11	57	36

Elites are not the only parents. Elitism decides which designs *survive* unchanged; selection decides which designs *reproduce*, and every individual can be chosen. In a binary tournament two individuals are drawn at random (with replacement) and the better one wins, so the individual of rank r (1 = best) wins a tournament with probability

$$\Pr(r) = \frac{2(P - r) + 1}{P^2}. \tag{9.4}$$

With the $P - n_e = 38$ tournaments of a generation of $P = 40$, about 22 different individuals (56 %) become parents. The best design is chosen 1.9 times on average, rank 10 1.5 times, rank 20 once, rank 30 0.5 times and the worst almost never; three quarters of all parent choices fall on the better half (Table 9.5). Good genes therefore spread through selection; the elites only have to make sure that the best design itself is never lost.

Table 9.5: Expected number of times an individual of rank r is chosen as a parent in one generation, $(P - n_e)\Pr(r)$ with Eq. (9.4), for $P = 40$, $n_e = 2$.

rank r	1	2	5	10	20	30	40
expected choices	1.88	1.83	1.69	1.45	0.97	0.50	0.02

Why a small elite share. One elite already guarantees that the best fitness never gets worse, so the best-so-far curve of the GA is monotone like that of SA; a second elite protects a second, different design, often one with another fleet size. More elites have two costs: every elite place is one new design fewer per generation, and the elites, which survive every generation *and* are chosen as parents about twice each, quickly fill the population with copies and near-copies of a few fleets (premature convergence). This matters here because many designs are almost equally good: the forced-fleet-size study (validation V6) showed that J changes little between two, three and four spacecraft, and a small elite share keeps these alternatives in competition.

Why separate crossover and mutation groups. Each operator has one job. Crossover *combines* what the population already holds: a child can inherit a good polar orbit from one parent and a good equatorial orbit from the other, a combination that SA can only reach step by step. Because whole slots are inherited, their simulations are already in the engine's cache and a crossover child is cheap to evaluate. Slot crossover cannot, however, create an orbit radius or an inclination that no parent has, and it changes the fleet size only by recombining switches; mutation *creates* new gene values and switches spacecraft on and off. The mutation children make the same 1–2-gene move as SA's neighbour function, so they can be read as SA-like local steps and the crossover children as what the GA adds on top. Keeping the groups separate also lets every new best design be traced to the operator that produced it.

Parameters to be studied. Both shares are treated as parameters and varied one at a time at the same budget (Table 9.6). The elite share is expected to show a U-shape: without elites the best design can be lost, with many elites the population converges early. A crossover fraction of 1 removes the mutation group, so no new gene values can appear after the first generation; a fraction of 0 leaves only mutation, which turns the GA into many SA-like moves in parallel without a temperature.

Table 9.6: Values of the elite share and the crossover fraction tested in the parameter study (one parameter changed at a time; default in bold).

elite share ε	0	2.5 %	**5 %**	10 %	20 %
elites n_e for $P = 40$	0	1	**2**	4	8
elites n_e for $P = 60$	0	2	**3**	6	12
generations G for $P = 40$	50	52	**53**	56	63
crossover fraction χ	0	0.6	**0.8**	0.95	1.0

9.6 Crossover

Crossover builds two children from two parents chosen by tournament. It works on whole slots, in three steps: the slots of the second parent are *aligned* to those of the first, each aligned pair is then either *swapped* or *blended*, and the children are repaired (Algorithm 2, Figure 9.3).

9.6.1 Step 1: slot alignment

The slots carry no identity: a fleet with orbits $(\mathcal{O}_1, \mathcal{O}_2, \mathcal{O}_3)$ in slots 1–3 is the same mission as one with the orbits in slots 3, 1, 2. With $N_{\max} = 5$ every fleet can be written in up to $5! = 120$ slot orders. Combining slot i of one parent with slot i of the other therefore usually pairs two *unrelated* orbits: a child may receive the same orbit twice (which then violates the separation constraint g3) or lose the orbits that made both parents good. Before crossover, the slots of parent B are therefore reordered so that each slot of parent A faces the most similar slot of B. The distance between two slots is

$$d(\mathbf{y}_i^A, \mathbf{y}_j^B) = \frac{|a_i^A - a_j^B|}{r_{\max} - r_{\min}} + \frac{|\iota_i^A - \iota_j^B|}{180^\circ} + \frac{\delta(\Omega_i^A, \Omega_j^B)}{180^\circ} + \frac{\delta(M_{0,i}^A, M_{0,j}^B)}{180^\circ}, \qquad \delta(\theta, \phi) = \min\big(|\theta - \phi| \bmod 360^\circ,\ 360 \tag{9.5}$$

where δ is the angle between two directions, so that 350° and 10° are 20° apart and not 340°. The pairs are matched greedily: flying slots of A with flying slots of B first, closest pair first; then the remaining flying slots with switched-off ones; finally the switched-off slots with each other. For $N_{\max} \leq 10$ this costs at most 100 distance evaluations, nothing compared with one mission simulation, and needs no optimisation package.

9.6.2 Step 2: swap or blend each aligned pair

Each aligned pair of slots is treated in one of two ways:

- **Swap** (probability $1 - p_b$): child 1 takes the whole slot from one parent, chosen with probability $\frac{1}{2}$, and child 2 from the other. The orbit is inherited intact, with its radius, inclination, node, phase and switch.
- **Blend** (probability p_b, only if *both* slots fly): the orbit genes are interpolated with a weight $\lambda \sim U(-\alpha_b,\, 1+\alpha_b)$ drawn per gene (blend crossover BLX-α),

$$g^{(1)} = g^A + \lambda\,(g^B - g^A), \qquad g^{(2)} = g^B + \lambda\,(g^A - g^B), \qquad g \in \{a, \iota\}, \tag{9.6}$$

and the angles along the shorter arc,

$$\Delta = \big((\theta^B - \theta^A + 180^\circ) \bmod 360^\circ\big) - 180^\circ, \qquad \theta^{(1)} = (\theta^A + \lambda\Delta) \bmod 360^\circ, \qquad \theta \in \{\Omega, M_0\}. \tag{9.7}$$

Both children keep the switch on.

The mission duration T is treated like a slot: swapped with probability $1-p_b$, otherwise blended with Eq. (9.6). A pair in which only one slot flies is always swapped, because an average of an orbit that flies and one that does not has no meaning; the swap passes the flying orbit to one child and the dormant one to the other, which is how crossover changes the fleet size. Finally both children are passed through `repair` (reflection of a, ι and T at their bounds, which BLX-α can exceed, and wrapping of the angles), and a child identical to a member of the population is mutated.

Why the angles need Eq. (9.7). The plain average of 350° and 10° is 180°, the opposite direction, although the two nodes are only 20° apart; the shorter-arc rule gives 0°. Without it, two nearly identical orbits would produce a child on the other side of the asteroid.

Why a random weight. With the fixed weight $\lambda = \frac{1}{2}$ (the midpoint of the two parents) every blended gene lies strictly between its parents, and without selection the spread of a gene in the population is halved each generation. After about five generations the population would be confined to roughly $1/30$ of its initial spread, long before the end of the run. With $\lambda \sim U(-\alpha_b, 1+\alpha_b)$ a child can also land slightly beyond either parent, which keeps the spread of the population; $\alpha_b = 0.3$ is used.

9.6.3 Choice of the blend probability

Swapping and blending have different roles. Swapping *recombines* complete orbits that have already proved good: an orbit is good as a whole (radius, inclination and node together), and its simulation is already in the engine's cache, so a swapped slot costs no new simulation.

Blending *refines*: it creates an orbit between two matched good orbits, e.g. a radius of 57 km between parents at 55 and 60 km, which swapping can never produce; each blended orbit is a new orbit to simulate. Swapping should therefore dominate, with enough blending to keep refining the orbits.

A crossover child has about $N \approx 3$ matched flying pairs (SA chose two to four spacecraft), so it contains on average $p_b N$ new orbits (Table 9.7). The default $p_b = 0.3 \approx 1/N$ gives about *one* new orbit per child, which matches the mutation rate (about one changed gene per child) and the cost of one SA move (one changed orbit, one new simulation), so that GA and SA also spend similar computing time per evaluation. The blend probability is a parameter of the study; a value found there replaces this default only if it is significantly better (Section 10.3).

Table 9.7: Effect of the blend probability p_b on a crossover child with $N = 3$ matched flying pairs. The values marked $\star$ are tested in the parameter study, together with the fixed weight $\lambda = \frac{1}{2}$ and with the alignment switched off.

p_b	$0^\star$	$\mathbf{0.3}^\star$	$0.5^\star$	$1^\star$
new (blended) orbits per child, $p_b N$	0	**0.9**	1.5	3
children without a new orbit, $(1-p_b)^N$	100 %	**34 %**	12.5 %	0 %
expected behaviour	no refinement; only mutation improves an orbit	recombination and refinement	more refinement, more simulations	every orbit new; population contracts

Algorithm 2 Slot crossover of two parents

Require: parents $\mathbf{x}^A, \mathbf{x}^B$; blend probability $p_b = 0.3$; $\alpha_b = 0.3$
Align: reorder the slots of $\mathbf{x}^B$ by greedy matching with Eq. (9.5) (flying–flying first)
$\mathbf{c}^{(1)} \leftarrow \mathbf{x}^A$; $\mathbf{c}^{(2)} \leftarrow \mathbf{x}^B$ (aligned)
for each gene group $k \in \{T,\ \text{slot } 1, \ldots, \text{slot } N_{\max}\}$ **do**
  if $\mathrm{rand}() < p_b$ **and** group k is T or flies in both parents **then**
    Blend: a, ι, T with Eq. (9.6); Ω, M_0 with Eq. (9.7); $\lambda \sim U(-\alpha_b, 1+\alpha_b)$ per gene
  else if $\mathrm{rand}() < \frac{1}{2}$ **then**
    Swap: exchange group k between $\mathbf{c}^{(1)}$ and $\mathbf{c}^{(2)}$ (whole slot, switch included)
  end if
end for
$\mathbf{c}^{(1)}, \mathbf{c}^{(2)} \leftarrow \textsc{Repair}(\mathbf{c}^{(1)}), \textsc{Repair}(\mathbf{c}^{(2)})$
mutate any child that equals a member of the population (e.g. a parent)
return $\mathbf{c}^{(1)}, \mathbf{c}^{(2)}$

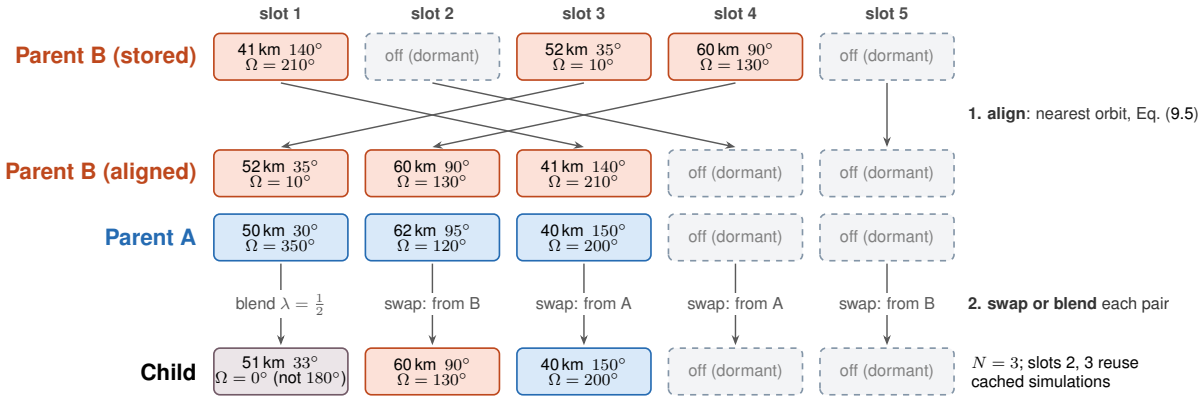


Figure 9.3: Slot crossover of two 3-spacecraft parents ($N_{\max} = 5$; the duration T is left out for clarity). Parent B stores its orbits in a different slot order; without alignment, slot 1 of A (50 km, 30°) would be paired with slot 1 of B (41 km, 140°), an unrelated orbit. After alignment each column holds two similar orbits. Here the first pair is blended (the nodes 350° and 10° give 0° along the shorter arc) and the others are swapped. Blue: from A; orange: from B; mixed: blended; grey: switched off.

9.7 Mutation

A mutation child is a copy of a parent in which one or two genes, chosen at random among the *active* genes (Section 9.1), are changed:

- a real gene g_j (T, a_i, ι_i, Ω_i or $M_{0,i}$ of a flying spacecraft) receives a Gaussian step proportional to its range,

$$g_j \leftarrow g_j + \mathcal{N}\big(0,\ (\sigma_g\,(u_j - l_j))^2\big); \tag{9.8}$$

- a switch s_i is flipped, which adds or removes a spacecraft. A slot that is switched on flies the dormant orbit it carries.

The child is then repaired. If repair maps it back onto the parent, or widens the change (switching off the last spacecraft makes repair switch another one on), the mutation is drawn again. This is the same move as the neighbour function of SA (Section 5.3), so the mutation children are SA-like local steps.

The step size σ_g used to breed generation $g+1$ from generation g shrinks geometrically, $g = 0, \ldots, G-2$,

$$\sigma_g = \sigma_0 \left(\frac{\sigma_{\min}}{\sigma_0}\right)^{g/(G-2)}, \qquad \sigma_0 = 0.25, \quad \sigma_{\min} = 0.01, \tag{9.9}$$

from a quarter of each variable's range (large jumps, exploration) to one percent (fine adjustment). In SA the step falls as $\sigma_0\sqrt{\Theta/\Theta_0}$ and the temperature falls geometrically, so its step

also shrinks geometrically from 0.25 to about 0.01; Eq. (9.9) gives the GA the same schedule, measured in generations instead of temperature levels.

Why only active genes. A change to a dormant gene does not alter the fitness, so it would cost an evaluation for nothing. Since every switch is an active gene, a switch is changed in about $N_{\max}/(1 + N_{\max} + 4N)$ of the gene choices (5 of 18 for $N = 3$, $N_{\max} = 5$), so the fleet size is mutated regularly.

Why one or two genes. A mutation child should stay close to its parent, a good design, so that the change can be judged: the effect of changing many genes at once is close to that of a random design. Changing one or two genes matches SA's move and, with the crossover children carrying about one new orbit each (Section 9.6), keeps the cost of the GA per evaluation close to that of SA.

9.8 Constraints, duplicates and termination

Constraints. Every child is evaluated through the same pipeline as in SA: a design that fails a pre-engine check (e.g. two spacecraft brought closer than d_{sep} by crossover) is not simulated and receives the penalised cost of Eq. (3.7). Such a child stays in the population: it loses every tournament against a feasible design, but among infeasible designs the smaller violation wins, so the search is led back to the feasible region. The separation and battery constraints cannot be repaired by moving genes into their bounds and are handled by this penalty only.

Duplicates. The elites are distinct designs, and a child identical to a member of the current population (its parents included) or to a child already created in the same generation is mutated again, so that no evaluation is spent on a design whose fitness is in the population. Designs of older generations are not stored: a child can in principle recreate one, but on the fleet problem this did not happen once in 4500 evaluations (three runs of 1500); on the low-dimensional benchmark functions (validation V2), where uniform crossover can rebuild an ancestor gene by gene, it costs 0.4–1.3 % of the budget.

Termination. The run stops when the evaluation budget B is used, as for SA; the last generation may be incomplete. No stall criterion is used, so both algorithms always spend the same number of evaluations. The result is the best design evaluated during the whole run.

9.9 One algorithm for the fleet and for benchmark functions

The GA is validated, like SA, on benchmark functions with a known optimum (validation V2). These functions have no slots and no switches, so the problem-specific operators are attached

to an optional part of the `Problem` interface: the fleet pipeline reports which genes form each slot (`slot_groups`) and which genes are switches. A problem without this information is treated with every gene as its own group: alignment is skipped, the slot swap becomes the standard uniform crossover, the blend becomes the standard BLX-α, and mutation changes one or two genes. The same code therefore runs on both kinds of problem, and the benchmark results test the GA machinery (selection, elitism, crossover, mutation, budget) independently of the spacecraft model.

9.10 The complete algorithm

Algorithm 3 and Figure 9.4 summarise the complete GA; Table 9.8 lists its parameters.

Algorithm 3 Genetic Algorithm for the fleet design

Require: budget B; P (Eq. 9.1); ε, χ (Eq. 9.3); p_b, α_b; k; σ_0, $\sigma_{\min}$

Start: draw P designs stratified over $N = 1, \ldots, N_{\max}$, each passing the pre-engine checks
evaluate them (Feasibility $\rightarrow$ Engine); $\mathbf{x}^* \leftarrow$ the best; $g \leftarrow 0$
$n_e, n_c, n_m \leftarrow$ Eq. (9.3); $G \leftarrow$ Eq. (9.2)
while the budget is not used **do**
  $\sigma_g \leftarrow$ Eq. (9.9); $\mathcal{E} \leftarrow$ the n_e best distinct designs (elitism)
  $\mathcal{C} \leftarrow \emptyset$
  while $|\mathcal{C}| < n_c$ **do**
    $\mathbf{x}^A, \mathbf{x}^B \leftarrow$ two different tournament winners
    add the children of CROSSOVER$(\mathbf{x}^A, \mathbf{x}^B)$ (Algorithm 2) to $\mathcal{C}$
  end while
  for n_m times **do**
    add REPAIR(MUTATE(tournament winner, σ_g)) to $\mathcal{C}$ (1–2 active genes)
  end for
  mutate again every child that duplicates a member of the population or another child
  evaluate the children until the budget is used; update $\mathbf{x}^*$ (feasible first, then lower J)
  population $\leftarrow \mathcal{E} \cup \mathcal{C}$; $g \leftarrow g + 1$
end while
return $\mathbf{x}^*$

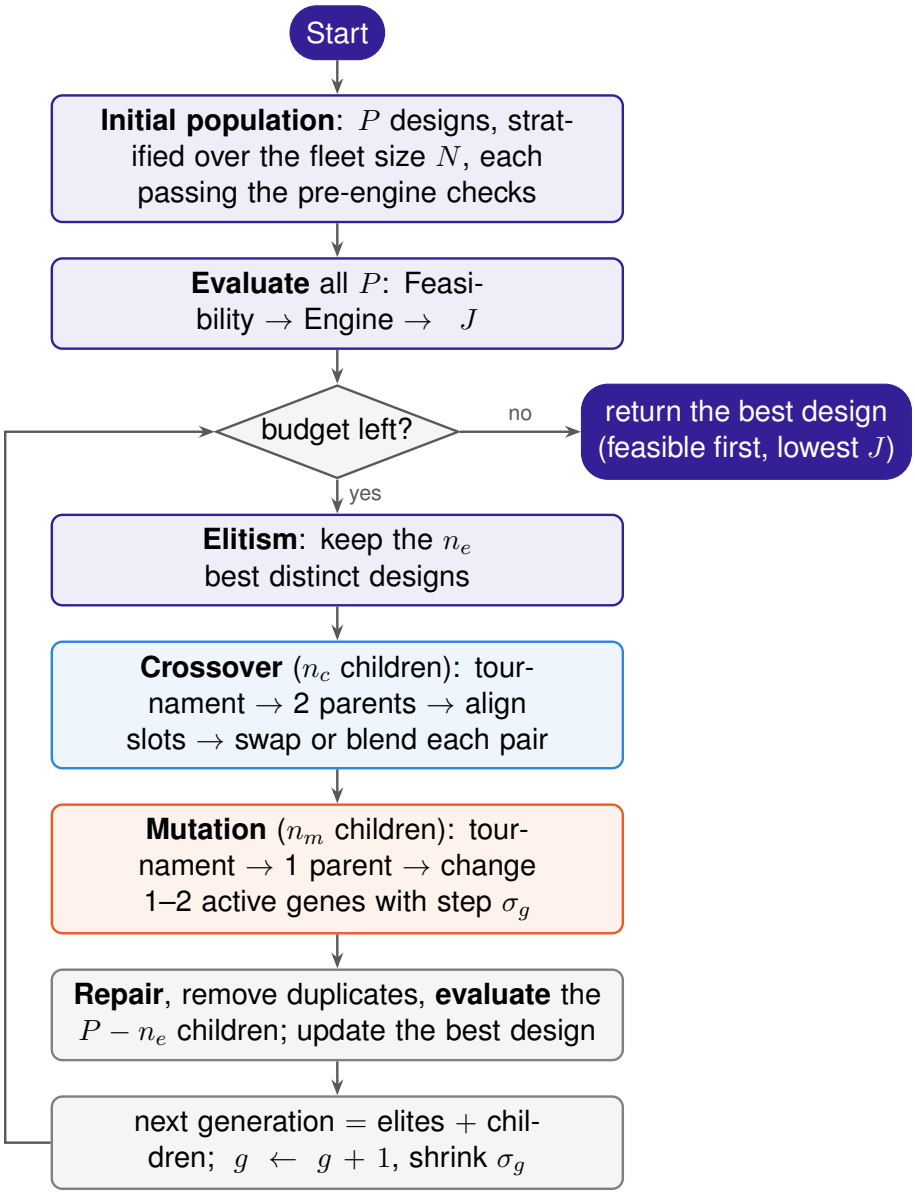


Figure 9.4: Flowchart of the Genetic Algorithm. Every evaluation passes through the same Feasibility → Engine pipeline as SA.

Table 9.8: Parameters of the GA: default values and the values compared in the parameter study (one parameter changed at a time; a value replaces the default only if it is significantly better, Section 10.3).

Parameter	Symbol	Default	Values in the study
Evaluation budget (per run)	B	2000	— (same as SA)
Population size	P	40 / 60 (Eq. 9.1)	10, 20, 40, 80, 120
Initial population	—	stratified	stratified, uniform
Elite share	ε	5 %	0, 2.5, 5, 10, 20 %
Crossover fraction	χ	0.8	0, 0.6, 0.8, 0.95, 1
Tournament size	k	2	2, 3, 5
Slot alignment	—	on	on, off
Blend probability	p_b	0.3	0, 0.3, 0.5, 1
Blend weight	λ	$U(-\alpha_b, 1+\alpha_b)$, $\alpha_b = 0.3$	random, fixed $\frac{1}{2}$
Genes changed per mutation	—	1 or 2	— (same as SA)
Initial / final mutation step (fraction of range)	σ_0 / $\sigma_{\min}$	0.25 / 0.01	$\sigma_0 = 0.1$, 0.25, 0.5
Engine cache (orbits)	—	1024	— (does not change results)

Chapter 10

Genetic Algorithm: Validation, Results and Comparison with SA

This chapter documents the GA case studies and inputs (Section 10.1), its generic validation (Section 10.2), the parameter study that fixed the settings of Table 9.8 (Section 10.3), the results of the case studies (Section 10.4), the harder tests (Section 10.5), the comparison with SA on the nine case studies (Section 10.6) and a matched comparison on a larger Eros case (Section 10.7). Every run is produced by `experiments/validate.py` (parts G2, GV4, GV5, GP, H, B, M and NI) and the report assets by `experiments/ga_assets.py`.

10.1 Case studies and input parameters

The GA is run on the nine case studies A1–A6 and B1–B3 of Table 6.1, with the *same* evaluation budget (2000), the *same* random seeds (0–4) and the same mission settings as SA, so that every GA run has an SA twin. Three new cases push the problem to its limits: the large body Kleopatra with very short missions, and a fleet of up to 20 spacecraft (101 genes, twice the largest case of Milestone 3). A fourth series repeats three cases with a budget of 5000 evaluations, and case M compares both algorithms once more on a ten-day Eros mission with ten slots and a finer search mesh. Table 10.1 lists the new cases; all GA parameters are those of Table 9.8.

Cases H1–H3 are also solved by random search. Case F changes one GA parameter at a time from the defaults (23 settings, Table 9.8).

Table 10.1: Additional case studies of this milestone (mission settings as in Table 6.1; $T_{\min} = \min(6\,\mathrm{h}, T_{\max}/4)$; five seeds per algorithm, three for case M).

Case	Body, setting	$T_{\max}$	$T_{\min}$	$N_{\max}$	dim	Budget	Question
H1	Kleopatra	6 h	1.5 h	10	51	2000	The hardest missions: a 219 km body
H2	Kleopatra	2 h	0.5 h	10	51	2000	with a few hours, up to 101 genes.
H3	Kleopatra	6 h	1.5 h	20	101	2000	Which algorithm copes better?
L1	Eros (= A2)	1 d	6 h	5	26	5000	Does a larger budget change the ranking of the two algorithms?
L2	Eros (= A6)	2 h	0.5 h	10	51	5000	
L3	Kleopatra (= B3)	12 h	3 h	10	51	5000	
F	Eros, GA parameters varied	2 d	6 h	5	26	2000	parameter study (same case as SA's case E)
M	Eros, 3 000-facet mesh	10 d	6 h	10	51	1200	matched comparison, seeds 0–2 (Section 10.7)

10.2 Validation

Known optima (G2). Since the GA only sees the `Problem` interface, it is run unchanged on the benchmark functions of validation V2, whose global minimum is 0 at $\mathbf{x} = 0$ (10 seeds, 10 000 evaluations each; Section 9.9: without slots, every gene is its own group). Table 10.2 compares it with SA. The GA reaches the optimum in every run of every function, with mean errors between 10^{-9} and 10^{-3} (at most $2 \cdot 10^{-3}$ in any run), several orders of magnitude below those of SA. On the five-dimensional Rastrigin function, which has about 10^5 local minima, SA ends below 0.1 in only 70 % of the runs and the GA in all of them: the population and the recombination of good coordinates from different individuals are exactly what a separable, highly multimodal function rewards. This validates the selection, elitism, crossover, mutation and budget machinery independently of the spacecraft model.

Table 10.2: Known-optimum test (minimum 0): best value reached, 10 seeds $\times$ 10 000 evaluations.

		mean best value		worst run		runs below 0.1
Function	n	SA	GA	SA	GA	SA / GA
Sphere	5	2.5e-03	6.8e-10	4.0e-03	6.1e-09	100 / 100 %
Rastrigin	2	2.1e-02	1.8e-08	1.2e-01	1.5e-07	90 / 100 %
Rastrigin	5	4.7e-01	3.9e-07	2.0e+00	2.3e-06	70 / 100 %
Ackley	2	1.2e-02	1.1e-04	2.4e-02	3.2e-04	100 / 100 %
Ackley	5	1.9e-02	3.2e-04	3.5e-02	1.9e-03	100 / 100 %

Comparison with the literature (GV5). Under the assumptions of the literature (every facet lit, no shadows; case D of Table 6.1) the GA images on average 97.7 % (best 97.9 %) of Eros,

against 97.9 % for SA (case D) and the 100 % that Nallapu and Thangavelautham [3] report with a simpler visibility model (validation V5). Both algorithms choose two spacecraft and a mission of about 1.16 days, and their fitness is the same within 0.5 % ($J = 0.0730$ for the GA, 0.0733 for SA).

Feasibility and finer model. All 45 final GA designs of the case studies are feasible. Re-evaluated on the four times finer shape model (6000 facets), the coverage of the 30 Eros designs changes by 2.0 points on average (at most 2.9), the same accuracy as found for the SA designs in validation V7: the GA does not exploit the coarseness of the search model.

10.3 Parameter study

Case F changes one parameter at a time from the starting values of Table 9.8 (five seeds each). Figure 10.1 shows every run and Table 10.3 the statistics. A value was to replace the starting value only if it was better by more than the seed-to-seed spread; a one-sided Mann–Whitney test [14] against the default decides.

Table 10.3: Parameter study (case F, five seeds per setting). Tracks: spacecraft actually simulated per run (computing cost independent of the machine).

Setting	mean J	std J	best J	mean C [%]	mean N	tracks / run
default	0.1685	0.0089	0.1608	92.7	3.0	1946
P 10	0.1755	0.0214	0.1553	92.3	2.8	1732
P 20	0.1879	0.0151	0.1704	93.4	3.0	1832
P 80	0.1730	0.0113	0.1633	93.3	3.0	1990
P 120	0.1869	0.0125	0.1689	90.4	3.2	2173
uniform start	0.1854	0.0159	0.1690	93.6	3.0	1784
elites 0 %	0.1725	0.0075	0.1621	92.0	3.4	2061
elites 2.5 %	0.1764	0.0134	0.1617	89.6	3.0	1938
elites 10 %	0.1768	0.0186	0.1593	93.7	3.0	1904
elites 20 %	0.1730	0.0111	0.1579	93.7	3.2	1918
crossover 0	0.1935	0.0116	0.1771	89.3	3.0	2127
crossover 0.6	0.1798	0.0147	0.1646	91.2	3.0	1947
crossover 0.95	0.1775	0.0089	0.1669	93.5	3.2	1826
crossover 1	0.1855	0.0105	0.1694	92.0	3.2	1936
tournament 3	0.1815	0.0157	0.1650	92.6	3.0	1866
tournament 5	0.1844	0.0125	0.1724	92.2	3.4	1894
no alignment	0.1847	0.0360	0.1620	92.0	3.2	1917
blend 0	0.1797	0.0166	0.1619	89.9	3.0	897
blend 0.5	0.1758	0.0083	0.1654	92.7	3.0	2895
blend 1	0.1676	0.0058	0.1608	92.0	2.8	5381
blend 1, $\lambda = \frac{1}{2}$	0.1989	0.0172	0.1826	90.1	3.2	4299
step0 0.10	0.1727	0.0105	0.1647	92.5	3.2	1913
step0 0.50	**0.1624**	0.0019	0.1600	93.7	3.2	1925

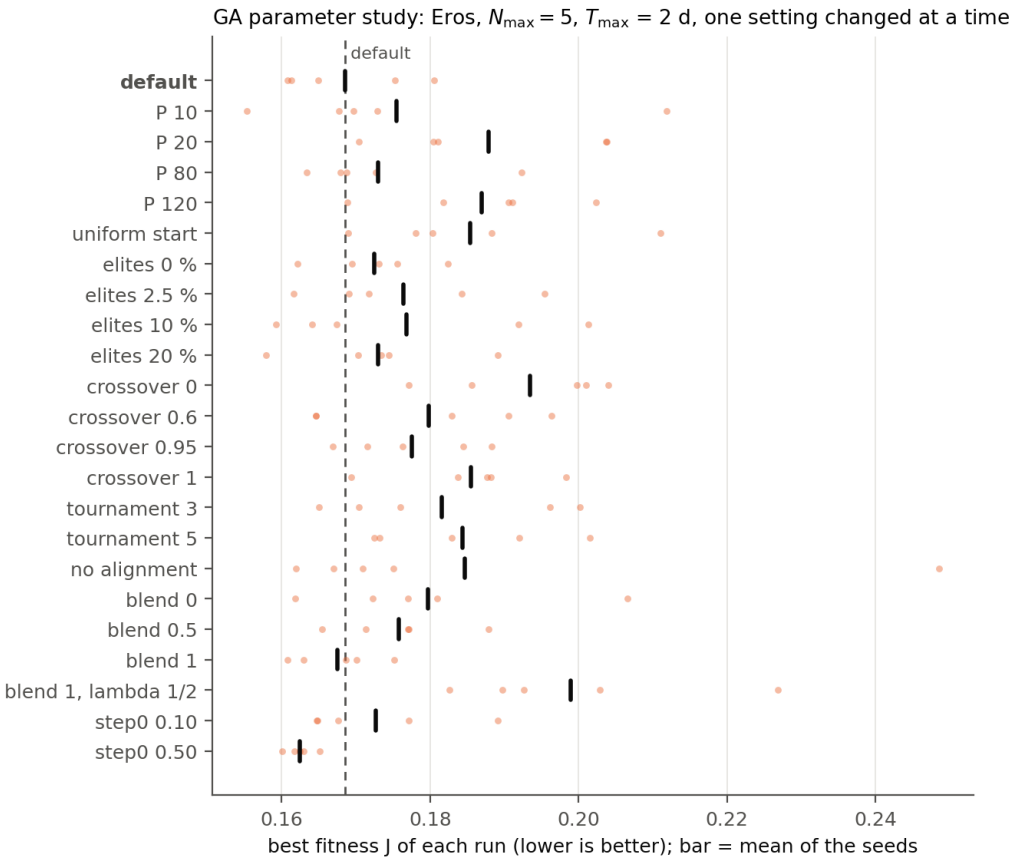


Figure 10.1: Parameter study of the GA (case F). Dots: best design of each of the five runs; bar: mean. Dashed line: the default settings.

- **No setting is significantly better than the defaults, and seven are significantly worse** ($p < 0.05$): $P = 20$, $P = 120$, the uniform start, a crossover fraction of 0 or 1, the tournament size $k = 5$ and the fixed blend weight $\lambda = \frac{1}{2}$. The defaults chosen by the reasoning of Chapter 9 are therefore kept.
- **Population size:** the expected U-shape. Small populations ($P = 10$, 20) lose diversity early; large ones ($P = 120$) leave only about 18 generations for selection and crossover to act.
- **Stratified start:** better than the uniform start by 0.017 ($p = 0.048$), as argued in Section 9.3.
- **Both operators are needed:** with mutation only ($\chi = 0$) the GA becomes a set of SA-like steps without SA's acceptance rule and is the second-worst setting; with crossover only ($\chi = 1$) no new gene values can appear. The mixture $\chi = 0.8$ is best.
- **Blend weight:** the midpoint $\lambda = \frac{1}{2}$ is the worst of all 23 settings ($p = 0.004$). It confirms the contraction argument of Section 9.6: every blended gene lies between its parents, the population shrinks and the search stalls.
- **Blend probability:** $p_b = 1$ reaches the same fitness as $p_b = 0.3$ but simulates 2.8 times more spacecraft (5381 against 1946 tracks per run), because every child orbit is new; $p_b = 0$ is the cheapest (897 tracks) but worse. $p_b = 0.3$ is the best compromise, as predicted by the "one new orbit per child" argument of Table 9.7.
- **Alignment:** without it the results become erratic (standard deviation four times that of the default): some runs pair unrelated orbits and fail.
- **Mutation step:** $\sigma_0 = 0.5$ had the best mean over the first five seeds (0.162). A tie-breaker with ten further seeds (5–14) for both values gave means of 0.1756 ($\sigma_0 = 0.5$) and 0.1786 ($\sigma_0 = 0.25$) over 15 seeds, $p = 0.53$: no real difference, so the default is kept. The tie-breaker also showed that five seeds can be optimistic: the default's mean rose from 0.169 to 0.179. Small differences between good settings in Table 10.3 are therefore within the noise; only the significant differences are interpreted.

10.4 Results of the case studies

Figure 10.2 shows the best design of every run of the nine case studies for the three algorithms, and Figure 10.3 the median convergence. Both optimisers beat random search by a wide margin in every case and every run. The GA reaches 73.5–95.7 % coverage on Eros missions of six hours to ten days and 47.2 % in two hours, and 64–76 % on Kleopatra.

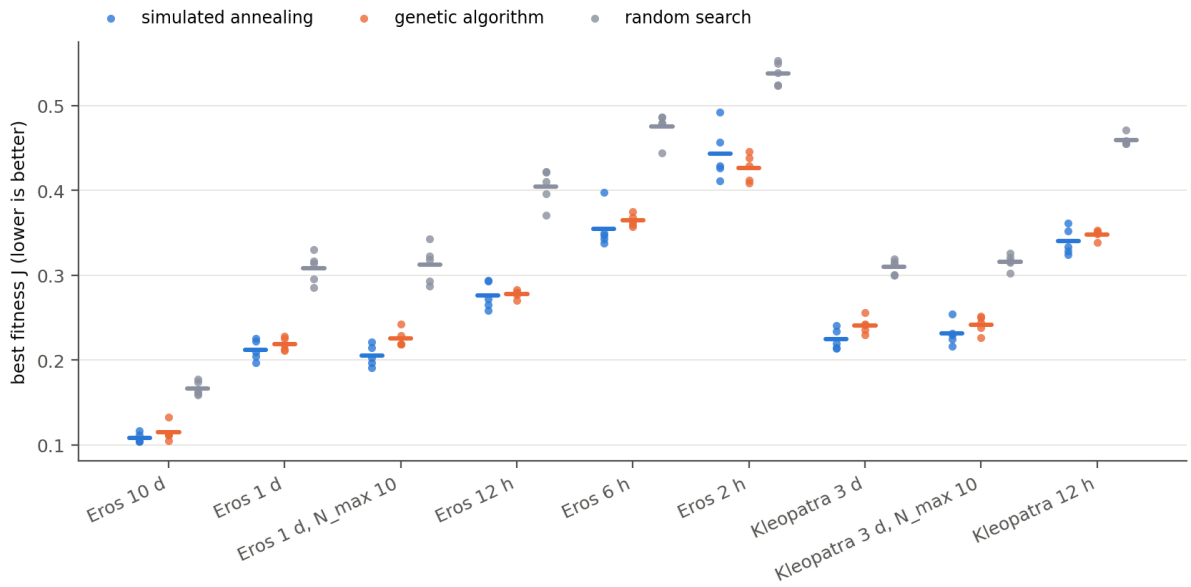


Figure 10.2: Best fitness of every run (dots) and mean (bar) for SA, GA and random search, budget 2000 evaluations, five seeds per algorithm and case.

Behaviour of one run. Figure 10.4 follows the GA run with the median result on Eros 10 d (case A1). The population mean improves quickly in the first 15 generations (from 0.42 to 0.14), while the diversity of the active genes (their spread as a share of the range; dormant genes of switched-off slots are not counted) falls from 0.31, the value of a random population, to about 0.03 after 20 generations: the population concentrates on one region and then refines it. The fleet sizes in the population stay mixed (two to four spacecraft) for most of the run; only in the last generations does the population settle on two spacecraft.

Which operator produces the improvements. Over all 45 case-study runs, the new best designs came from blend crossover in 73 % of the cases, from slot swaps in 18 % and from mutation in 9 % (the initial random designs excluded; Figure 10.4, right). Although blending is possible only for pairs of flying slots and is chosen for 30 % of them, it is the operator that *refines* orbits towards the optimum; swaps and mutation supply the combinations and the new values it works on. This is consistent with the parameter study, in which removing either the crossover or the mutation group made the GA significantly worse.

10.5 Harder tests: large body, short missions, larger budget

Hard cases H1–H3. Table 10.4 and Figure 10.5 (left) show the three hardest missions. On the two 10-slot missions H1 and H2 the GA has the lower mean fitness (by 1.5 and 1.3 %) and images 5.6 and 7.9 points more of Kleopatra than SA, but neither difference is significant.

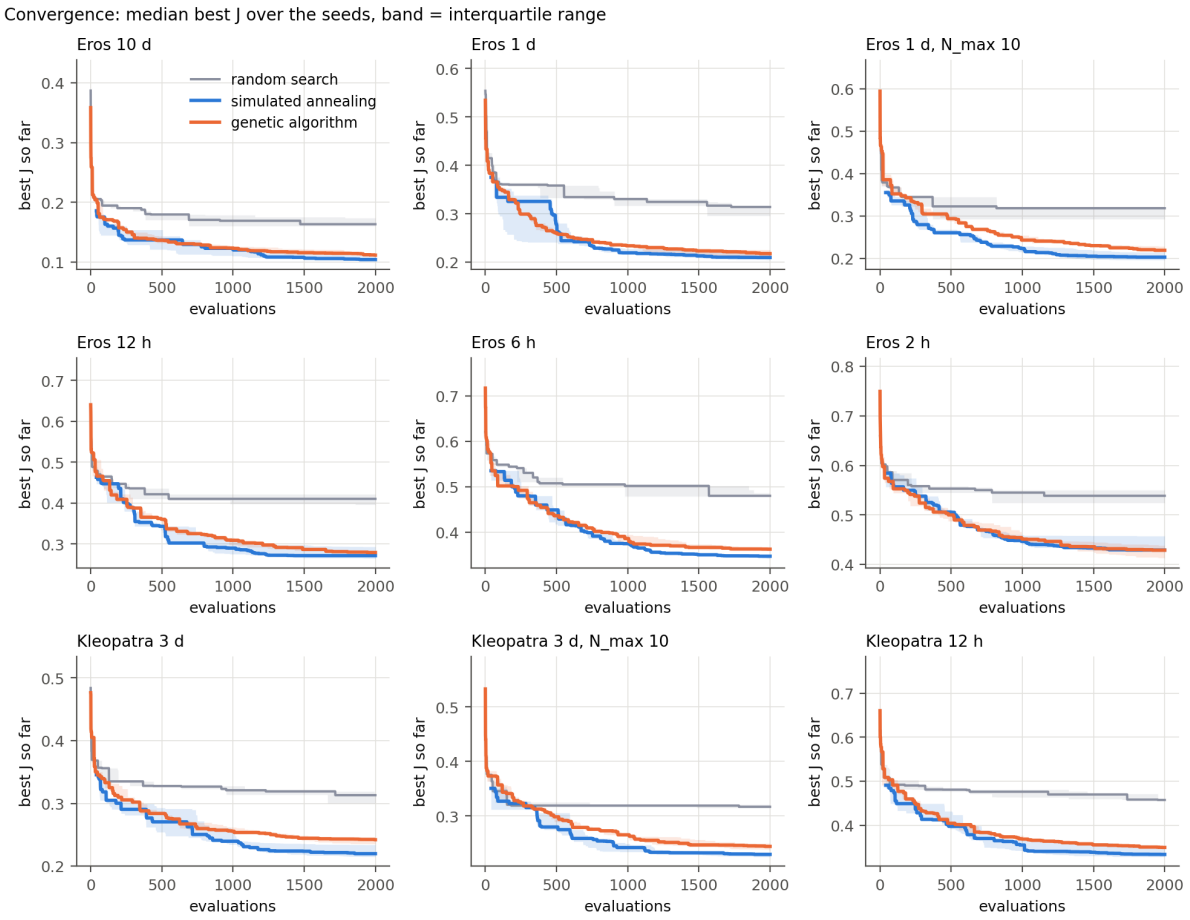


Figure 10.3: Convergence: median over the seeds of the best fitness found so far, with the interquartile band. SA's curve starts after its warm start (the first 40 evaluations are not recorded).

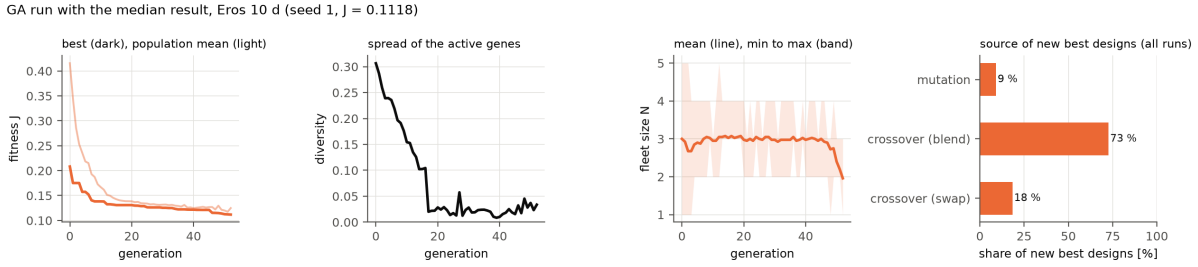


Figure 10.4: The GA run with the median result on Eros 10 d, generation by generation: fitness, diversity of the active genes, fleet sizes in the population; right: the operator that produced each new best design, over all 45 case-study runs.

On the 101-gene case H3 SA has the lower mean fitness (0.418 against 0.435; $p = 0.11$ for SA $<$ GA, not significant), and both algorithms reduce the fleet to three or four spacecraft in every run, while random search still sends 3 to 8. A first run of H3 used a cost normalisation $n_{\mathrm{ref}} = \max(10, N)$ that capped Z at one for fleets above ten spacecraft; under it three SA runs stayed at 14–15 spacecraft. With the fixed reference $n_{\mathrm{ref}} = 10$ of the final code (Eq. 3.4) every extra spacecraft costs $\Delta Z = 0.1$ at any fleet size, and the results above are those of the final code.

Table 10.4: Hard cases H1–H3 (2000 evaluations, five seeds): mean fitness, coverage and fleet size; p: one-sided Mann–Whitney test that the GA is better.

	mean J			mean C [%]			mean N		
Case	SA	GA	random	SA	GA	random	SA	GA	p (GA $<$ SA)
Kleopatra 6 h	0.4408	0.4343	0.5251	39.8	45.4	30.3	2.2	3.2	0.42
Kleopatra 2 h	0.4529	0.4469	0.5325	34.3	42.2	14.9	2.6	3.6	0.50
Kleopatra 6 h, $N_{\max} = 20$	0.4180	0.4353	0.5686	52.2	45.7	25.9	3.4	3.6	0.92

Larger budget L1–L3. With 5000 instead of 2000 evaluations both algorithms improve, but the GA 1.6 to 2.9 times more (Table 10.5, Figure 10.5, right). Where SA led at 2000 evaluations the gap shrinks from 0.007 to 0.002 (Eros 1 d) and from 0.008 to 0.005 (Kleopatra 12 h); on the two-hour Eros mission, where the GA already led, its advantage becomes significant ($p = 0.048$).

Table 10.5: Mean fitness with 2000 and 5000 evaluations (same seeds); p: one-sided Mann–Whitney tests at 5000 evaluations (bold: $p < 0.05$).

	SA mean J		GA mean J		p at 5000	
Case	2000	5000	2000	5000	GA $<$ SA	SA $<$ GA
Eros 1 d	0.2119	0.2092	0.2191	0.2113	0.790	0.274
Eros 2 h	0.4432	0.4371	0.4268	0.4133	**0.048**	0.972
Kleopatra 12 h	0.3403	0.3357	0.3482	0.3410	0.845	0.210

10.6 Comparison of SA and GA on the case studies

The two algorithms are compared with six performance indices, four of which are specific to the fleet problem (Table 10.6):

1. **Fitness** J (mean and spread over the seeds, one-sided Mann–Whitney test);
2. **Coverage** C of the sunlit surface, the mission's actual product;
3. **Resources** $N \cdot T$ in spacecraft-days, what the mission occupies;

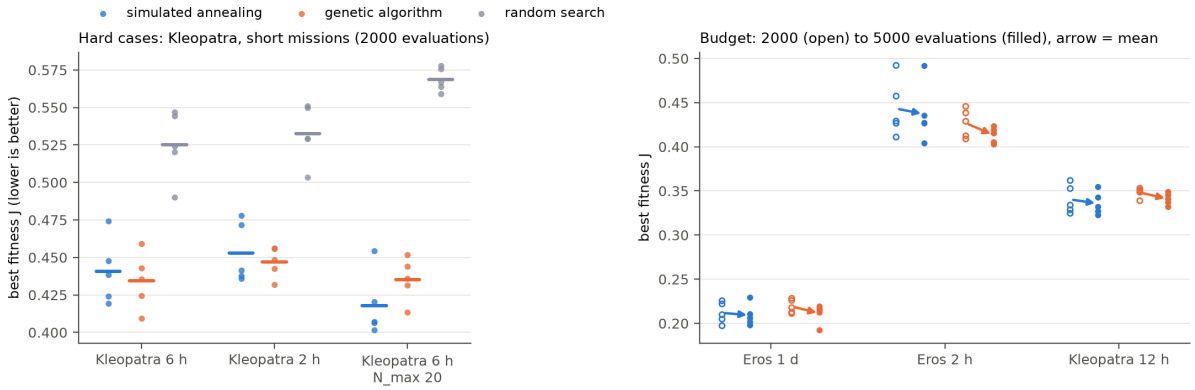


Figure 10.5: Left: hard cases H1–H3, best fitness of every run and mean. Right: the same seeds with a budget of 2000 (open markers) and 5000 evaluations (filled); the arrow joins the means.

4. **Convergence:** evaluations needed until the best fitness first reaches a target, the worse of the two algorithms' median final fitness;
5. **Computing cost:** spacecraft actually simulated per run (*tracks*, independent of the machine);
6. **Reliability:** the standard deviation and the worst run over the seeds.

Table 10.6: SA against GA on the nine case studies (2000 evaluations, five seeds, means). Bold: significantly better ($p < 0.05$). Evals to target: median evaluations until the best fitness reaches the worse of the two median final values.

	mean J		mean C [%]		$N \cdot T$ [d]		evals to target		tracks	
Case study	SA	GA	SA	GA	SA	GA	SA	GA	SA	GA
Eros 10 d	0.1080	0.1148	95.1	95.7	4.43	5.32	1035	1920	1848	1704
Eros 1 d	0.2119	0.2191	88.6	86.6	1.31	1.35	940	1377	2009	1970
Eros 1 d, $N_{\max} = 10$	**0.2052**	0.2256	89.6	89.4	1.31	1.81	960	1914	1871	2725
Eros 12 h	0.2766	0.2780	80.5	81.9	0.90	1.02	871	1799	1893	2716
Eros 6 h	0.3552	0.3646	75.0	73.5	0.63	0.69	1120	1573	1826	2187
Eros 2 h	0.4432	0.4268	38.7	47.2	0.09	0.13	1493	1386	1638	1926
Kleopatra 3 d	**0.2247**	0.2412	74.8	75.6	1.80	2.68	858	1481	1938	1860
Kleopatra 3 d, $N_{\max} = 10$	0.2315	0.2420	76.3	76.4	2.43	2.88	822	1422	1785	2382
Kleopatra 12 h	0.3403	0.3482	61.6	64.3	0.61	0.84	774	1744	1791	2283

- **Fitness.** SA has the lower mean J in eight of the nine cases, by 0.5–9.9 % (mean J 0.2663 against 0.2734). The difference is significant in 2 cases (Eros 1 d with $N_{\max} = 10$ and Kleopatra 3 d) and within the seed-to-seed spread in the other 7; the GA is better on the two-hour Eros mission, the hardest of the nine.
- **Coverage and resources.** The GA sends more spacecraft in all nine cases (by 0.2 to 1.0 on average) and reaches equal or higher coverage in six. Its slightly higher J is the fleet cost of these extra spacecraft, not worse imaging. The forced-size study V6 showed that the fitness is nearly flat between two and four spacecraft, so both choices lie in the same

flat optimum; the Milestone 3 finding that SA tends to send slightly too few spacecraft where the fitness is flat in N is mirrored by the GA sending slightly more.

- **Convergence.** SA reaches the target in a median of 1.7 times fewer evaluations (range 0.9–2.3). This is the expected behaviour: a single chain improves from its first move, whereas the GA spends its first generations exploring with whole populations.
- **Computing cost.** Per evaluation both algorithms simulate about one new spacecraft (1700–2000 tracks per 2000 evaluations with $N_{\max} = 5$), so a run costs about the same. With $N_{\max} = 10$ the GA simulates up to 46 % more tracks, because its fleets are larger.
- **Reliability.** The GA's results vary less from seed to seed: the mean standard deviation of J is 0.009 against 0.016 for SA, and its worst run is better than SA's worst in five of the nine cases.

Overall. With the budget of 2000 evaluations, SA is slightly better and reaches the target fitness in about 1.7 times fewer evaluations on the standard missions. The GA is statistically on a par with it in seven of nine cases, varies less between seeds, has the lower mean fitness on the two-hour Eros mission and on the two short 10-slot Kleopatra missions, and profits more from a larger budget: with 5000 evaluations it closes most of the gap and becomes significantly better on the hardest Eros mission. At 101 genes both find small fleets and SA is slightly ahead. The choice therefore depends on the problem: SA for small budgets and moderate missions, the GA for very short missions and larger budgets.

10.7 Matched comparison on a larger Eros case

Case M repeats the comparison once more on a different configuration: Eros with a 3 000-facet search mesh (instead of the 1 499 facets of the case studies), a ten-day mission horizon, $N_{\max} = 10$ (51 genes), full lighting, cast shadows and terrain occlusion, the objective weights $(w_C, w_Z, w_E, w_T) = (0.50, 0.15, 0.15, 0.20)$ and $n_{\mathrm{ref}} = 10$. Each algorithm receives the same 1 200 evaluations for each of seeds 0, 1 and 2 (part M of `validate.py`).

SA obtains the lower mean fitness, $J = 0.1089$, against $J = 0.1237$ for the GA, and the higher mean coverage, 94.44 % against 93.53 %. SA also uses fewer spacecraft on average (2.33 against 3.00) and its runs are faster, because the GA's children contain more new orbits to simulate (about 1 500 against 1 100 tracks per run). The GA produces a slightly shorter mean mission (1.77 against 1.84 days). All 3 GA and 3 SA final designs are feasible.

The result agrees with the case studies: with a budget of 1200–2000 evaluations, SA exploits a promising design immediately through local moves, whereas the GA invests evaluations in maintaining and recombining a population. A hybrid strategy that applies SA refinement to the

Table 10.7: Matched comparison of GA and SA on Eros (case M, 1200 evaluations each). Lower J is better; all final designs are feasible. Runtime: wall-clock seconds of one run with nine runs in parallel on the same machine.

Seed	Algorithm	J	Coverage [%]	T [d]	N	Runtime [s]
0	GA	0.1159	92.44	2.12	2	217.89
0	SA	0.1034	94.54	1.86	2	151.55
1	GA	0.1229	92.86	1.67	3	209.41
1	SA	0.1031	95.89	1.38	3	169.77
2	GA	0.1323	95.28	1.52	4	214.48
2	SA	0.1203	92.90	2.28	2	153.01
GA mean		0.1237	93.53	1.77	3.00	213.93
SA mean		0.1089	94.44	1.84	2.33	158.11

best GA individual, together with parallel child simulation, is therefore the most direct next improvement.

10.7.1 Evidence across the complete comparison set

Table 10.8 consolidates all GA–SA evidence generated for this milestone. The standard study contains nine spacecraft cases, with five matched seeds and the same evaluation budget for both algorithms. The hard-case study adds short Kleopatra missions and a 101-variable fleet design. The budget sweep tests whether the ranking changes when the budget is increased from 2,000 to 5,000 evaluations. The mathematical benchmark and all-lit studies isolate algorithm behaviour from parts of the spacecraft physics.

The two Eros experiments of case M must be read together. Equal evaluations answer which method uses expensive simulations more efficiently. The equal-native-iterations run only shows what 20 GA generations achieve: one GA generation contains a population of evaluations, whereas the 20 SA evaluations are spent on the warm start before the first SA move, so this run says nothing about SA's annealing. Neither experiment alone establishes universal superiority.

10.7.2 Why mutation and crossover are different

Mutation and crossover are deliberately different because they solve two different search problems. Crossover is primarily a recombination operator. It takes two already evaluated parents and transfers useful structures between them. In this project, slot alignment makes a spacecraft orbit in one parent correspond to a similar orbit in the other parent. Swap crossover preserves a complete orbit, while blend crossover creates a nearby continuous orbit between the two parents. Crossover can therefore combine a good duration, fleet composition and orbit arrangement discovered in different individuals.

Table 10.8: Complete set of GA–SA comparisons.

Study	Configuration	Budget	Main finding
Standard cases	Nine Eros/Kleopatra missions; five matched seeds	2,000	SA has the lower mean J in eight of nine cases (0.2663 against 0.2734), significantly in 2; the GA is better on Eros 2 h, sends slightly larger fleets and varies less between seeds.
Hard cases	Kleopatra, 6 h; Kleopatra, 2 h; Kleopatra, 6 h with $N_{\max} = 20$ and 101 genes	2,000	The GA has the lower mean J on the two 10-slot missions and images 6–8 points more, not significantly; on the 101-gene case SA is slightly better (0.418 against 0.435, not significant) and both send three or four spacecraft. Random search is far behind in every case.
Budget sweep	Eros 1 d, Eros 2 h and Kleopatra 12 h	2,000 / 5,000	Both improve, the GA 1.6–2.9 times more; it closes most of the gap and becomes significantly better on Eros 2 h ($p = 0.048$).
Known optima	Sphere, Rastrigin and Ackley benchmark functions; ten seeds	10,000	The GA reaches every optimum in every run; it is especially reliable on the multimodal Rastrigin function.
All-lit model	Eros with sunlight and cast shadows switched off	2,000	Reproduces the simplified literature-style setting; GA and SA are nearly tied in coverage and objective.
Equal evaluation budget (case M)	Eros, 3,000 facets, $N_{\max} = 10$; three seeds	1,200 each	SA obtains mean $J = 0.1089$ against 0.1237 for the GA and is faster under the same simulation budget.
Equal native iterations (part NI)	Case M; three seeds	20 GA generations vs. 20 SA evaluations	The GA obtains mean $J = 0.1321$ with 1143 evaluations. SA's 20 evaluations are all used by its warm start (best of $n_{\text{init}} = 20$ random designs), so no annealing move is made and its $J = 0.2177$ is that of random sampling; this is not an equal-cost comparison and not a test of SA's search.

Mutation is primarily an exploration and local-refinement operator. It changes one or two active genes of a selected parent, such as mission duration, radius, inclination or angle. It can introduce a value absent from both parents and move the population into a new region. It also helps replace duplicate children and preserve population diversity.

The distinction is visible in the parameter study. With crossover disabled, the population becomes a collection of mutation-driven local moves and loses the ability to combine complete solutions. With mutation disabled, selection and crossover increasingly reuse existing genetic material; the population can contract and may fail to introduce a useful value or escape a converged region. The selected mixture, $\chi = 0.8$ for crossover and the remaining 20% for mutation, therefore provides both recombination and new-value generation. Within a crossover, each aligned pair of flying slots (and the duration T) is blended with probability $p_b = 0.3$; otherwise, and always for a pair in which only one slot flies, each child takes the whole slot from one of the parents, chosen with probability $\frac{1}{2}$ (Algorithm 2). These probabilities are distinct from the population-level crossover fraction.

Mutation is not simply a second form of crossover. Crossover asks, "Which useful parts of two existing designs can be combined?" Mutation asks, "Which new nearby design should be tested?" The GA needs both mechanisms, while SA supplies mutation-like moves together with a probabilistic acceptance rule. A practical hybrid can use GA crossover for global recombination and SA-like local refinement for the final search stage.

10.8 Recommendations

- **Hybrid search:** reserve the final part of the GA budget for SA-like local refinement around the best individual, combining the GA's exploration with SA's fine-tuning, the weakness seen in the convergence index.
- **Parallel evaluation:** the $P - n_e$ children of a generation are independent and can be simulated at the same time; on a multi-core machine the GA would then be several times faster in wall-clock time, while SA's chain is inherently sequential.
- **More seeds and sensitivity analysis:** the tie-breaker showed that five seeds can mislead by about 0.01 in J; repeat the comparison over objective weights and ten or more seeds before selecting an operational default.

Chapter 11

Conclusion

- **Formulation and implementation.** The fleet design problem was finalised as a mixed-integer, single-objective problem in which the optimiser chooses the number of spacecraft (on/off switches), the circular orbit of each, and the mission duration, trading coverage of the sunlit surface against fleet cost, energy and time under radius, separation and battery constraints. It is implemented as a modular Algorithm $\rightarrow$ Feasibility $\rightarrow$ Engine pipeline around a true-scale simulation (Kepler orbits, spinning non-convex shape model, occlusion, sunlight and shadows, camera resolution, batteries). Both SA and GA use this same pipeline; their search operators remain separate from the physics model.
- **Designs.** SA sends only the spacecraft a mission needs: two for long Eros missions, which it ends after about two days at 95 % coverage, and one to five (about three on average) when only hours are available. The same code and settings work unchanged on the 46 times larger, strongly concave Kleopatra (two to three spacecraft, 75 % in about 18 h). The optimiser discovered on its own that the best orbits sit just above the camera's range, at 3.5 times the body's radius on both bodies, and are tilted about 30–35° rather than polar because the poles are never lit.
- **Validation.** The simulator agrees with hand calculations; SA finds the known optima of benchmark functions; it beats random search with the same budget in all nine settings and in every run ($p = 0.004$), by 6 to 21 points of coverage, the more so the shorter the mission; under the assumptions of the literature it images 98 % of Eros, close to the 100 % reported there; its fleet sizes lie in the flat optimum found by forcing the fleet size; and its designs hold on a four times finer model.
- **Algorithm.** The cooling schedule is the most important parameter: fitting α to the budget avoids both premature freezing and a search that never settles. A moderate start temperature ($p_0 \leq 0.5$) and large initial steps are best. The track cache makes a move cost at most two spacecraft simulations, so SA is also two to four times faster per evaluation

than random search.

- **Genetic Algorithm (Milestone 4).** The GA uses the whole mission as chromosome, aligns spacecraft slots before crossover, swaps or blends complete orbits, starts with a population stratified over fleet size and reuses SA's local move as mutation. A 23-setting parameter study confirmed the default design choices and showed that both operator families are needed. On the nine case studies SA is slightly better (mean J 0.2663 against 0.2734, significant in 2 cases) and converges faster, while the GA is on a par in 7 cases, varies less between seeds, has the lower mean fitness on the shortest Eros and Kleopatra missions and profits more from a larger budget (significantly better on Eros 2 h with 5000 evaluations). In the matched 51-variable Eros case, SA achieved mean $J = 0.1089$ and 94.44 % coverage, compared with $J = 0.1237$ and 93.53 % for the GA; SA was faster and used fewer spacecraft on average, while the GA produced a shorter mean mission. A hybrid local-refinement stage is the clearest next improvement for the GA.
- **Limitations.** The orbits ignore the irregular gravity field; the Sun direction is fixed; the weights decide the value of one more spacecraft or day; and the current comparisons remain based on a finite number of random seeds (Chapter 12).

Chapter 12

Future Work

- **Further algorithms.** The GA of this milestone plugs into the pipeline unchanged; the next milestone adds a swarm method (PSO) the same way, compared on the same case studies with the same budget, seeds and statistical tests.
- **Hybrid and parallel GA.** A final local search with SA's move around the best design would add SA's fine-tuning to the GA's exploration, and evaluating the children of a generation in parallel would make the GA several times faster in wall-clock time.
- **Trade-off without weights.** The weights decide how much one more spacecraft or one more day is worth. NSGA-II will return the whole Pareto front of coverage, fleet cost, energy and mission time, on which the weighted designs of this milestone should lie.
- **Gravity.** Orbits are ideal Kepler circles. The irregular gravity of Eros, and even more of the dog-bone shaped Kleopatra, makes close orbits drift or become unstable; a gravity field computed from the shape (polyhedron model) is the next step towards realism.
- **Seasons and pointing.** The Sun direction is fixed and the cameras always point at the centre of mass; a moving Sun and off-nadir pointing would add realism and decision variables.
- **Statistics.** Some forced-size cases use three runs; more runs would tighten the spreads and the p-values.
- **Reproducible validation.** Every change to the objective, constraints or mesh should trigger a regeneration of the comparison matrix with the same seeds and locked dependencies. The validation output should retain the git commit, package versions, mesh resolution, objective weights and reference fleet size alongside each result.
- **Sensitivity of the trade-off.** The fixed-reference normalization correctly penalises fleets larger than ten, but the preferred fleet still depends on the weights. A systematic sensi-

tivity study, followed by a Pareto-front analysis, would quantify how robust the selected mission is to the relative value assigned to coverage, cost, energy and duration.

Appendix

Appendix A

Running the Code

The code is written in Python 3 (3.9 or newer; the validated environment of `requirements-lock.txt` is Python 3.11). It uses NumPy for all numerics (the SA and GA algorithms themselves use nothing else), `trimesh` with Embree for ray casting, SciPy only for the Mann–Whitney test of the validation, Matplotlib for charts, and PyVista with PyQt5 for the 3D viewer. Every command is run from the project folder with the launcher `sim` (`sim.bat` on Windows), which creates a virtual environment and installs `requirements.txt` on first use.

Table A.1: Commands

Command	What it does
`./sim`	open the 3D viewer on the example design (optimiser available in the "Optimiser" tab)
`./sim eval scenarios/orbital_example.json`	print J, its terms and every constraint of a design
`./sim optimise scenarios/orbital_example.json --evals 2000 --seed 1 --body eros --n-max 5 --t-max 2 --save best.json`	run SA without the viewer and save the best design
`./sim optimise scenarios/orbital_example.json --algorithm ga --evals 2000 --seed 1 --save best.json`	the same with the GA (`--algorithm random`: random search)
`./sim best.json`	open a saved design in the viewer
`./sim validate --seeds 5 --evals 2000`	every case study and validation run of this report (V2–V7)
`./sim validate --only P`	the parameter study (case E)
`./sim validate --only G2 GV4 GV5 GP H B`	the GA validation, case studies, parameter study (case F), hard cases and budget sweep
`./sim validate --only M NI`	the matched GA–SA comparison and the equal-native-iterations run (fixed seeds and budgets)
`./sim test`	the 90 automated tests
`python report/LaTeX/make_assets.py`	rebuild the tables and charts of the SA chapters from the results
`python experiments/ga_assets.py`	rebuild the tables, charts and quoted numbers of the GA chapter

The validation writes to `results/validation/`: `run.log` (the console output, one line per run), `runs_partial.jsonl` (every run as soon as it finishes), `raw.json` (all runs with their best decision vector and convergence curve), `summary.json` and one CSV file per part, and `run_info.json` (command, git commit, package versions, machine, start and end time of the latest run; a copy named after the parts run, e.g. `run_info_H_M_NI.json`, is kept

for every run).

Appendix B

Decision Vector Layout

Table B.1: Layout of the decision vector $\mathbf{x}$ (Eq. 3.1); slot $i = 1 \dots N_{\max}$

Index	Variable	Lower bound	Upper bound	Kind
0	T (days)	$T_{\min}$	$T_{\max}$	real, reflected
$1 + 5(i-1)$	a_i (km)	$r_{\min}$	$r_{\max}$	real, reflected
$2 + 5(i-1)$	ι_i (deg)	0	180	real, reflected
$3 + 5(i-1)$	Ω_i (deg)	0	360	real, periodic
$4 + 5(i-1)$	$M_{0,i}$ (deg)	0	360	real, periodic
$5 + 5(i-1)$	s_i	0	1	binary

Bibliography

[1] David W. Dunham, Robert W. Farquhar, James V. McAdams, et al. Implementation of the first asteroid landing. *Icarus*, 159(2):433–438, 2002.

[2] Ravi Teja Nallapu and Jekan Thangavelautham. Automated design architectures for co-orbiting spacecraft swarms for planetary moon mapping. *Advances in Space Research*, 67(11):3559–3582, 2021.

[3] Ravi Teja Nallapu and Jekan Thangavelautham. Attitude control of spacecraft swarms for visual mapping of planetary bodies. In *Proceedings of the IEEE Aerospace Conference*, pages 1–16, Big Sky, MT, USA, 2019.

[4] Shimeles Demissie Melaku and Hae-Dong Kim. Optimization of multi-mission CubeSat constellations with a multi-objective genetic algorithm. *Remote Sensing*, 15(6):1572, 2023.

[5] Meiqian Guan, Tianhe Xu, Fan Gao, Wenfeng Nie, and Honglei Yang. Optimal Walker constellation design of LEO-based global navigation and augmentation system. *Remote Sensing*, 12(11):1845, 2020.

[6] Daniel Stolfi Rosso and Grégoire Danoy. Spacecraft swarm orbital formation optimisation using evolutionary techniques. In *Proceedings of the Genetic and Evolutionary Computation Conference Companion (GECCO '23)*. ACM, 2023.

[7] Carol A. Raymond, Ralf Jaumann, Andreas Nathues, et al. The Dawn topography investigation. *Space Science Reviews*, 163:487–510, 2011.

[8] Miroslav Brož, Franck Marchis, Laurent Jorda, et al. An advanced multipole model for (216) Kleopatra triple system. *Astronomy & Astrophysics*, 653:A56, 2021.

[9] Robert W. Gaskell. Gaskell Eros shape model v1.0. NASA Planetary Data System, NEAR-A-MSI-5-EROSSHAPE-V1.0, 2008.

[10] Steven J. Ostro, R. Scott Hudson, Michael C. Nolan, et al. Radar observations of asteroid 216 Kleopatra. *Science*, 288(5467):836–839, 2000.

[11] Tomas Möller and Ben Trumbore. Fast, minimum storage ray-triangle intersection. *Journal of Graphics Tools*, 2(1):21–28, 1997.

[12] Scott Kirkpatrick, C. Daniel Gelatt, and Mario P. Vecchi. Optimization by simulated annealing. *Science*, 220(4598):671–680, 1983.

[13] Nicholas Metropolis, Arianna W. Rosenbluth, Marshall N. Rosenbluth, Augusta H. Teller, and Edward Teller. Equation of state calculations by fast computing machines. *The Journal of Chemical Physics*, 21(6):1087–1092, 1953.

[14] Henry B. Mann and Donald R. Whitney. On a test of whether one of two random variables is stochastically larger than the other. *The Annals of Mathematical Statistics*, 18(1):50–60, 1947.

[15] John H. Holland. *Adaptation in Natural and Artificial Systems*. University of Michigan Press, Ann Arbor, MI, 1975.

[16] David E. Goldberg. *Genetic Algorithms in Search, Optimization, and Machine Learning*. Addison-Wesley, Reading, MA, 1989.